

legoESM: A Differentiable Earth System Model

Technical Specification v3.8

Project: legoESM **License:** MIT **Authors:** Pierre Gentine + Claude **Date:** 2026-03-23 (updated from v3.7, 2026-03-18)

Changelog (v3.8, 2026-03-23)

Production hardening, operationalization, and full test-suite stabilization:

- Full test suite stabilization** (35 → 0 failures): Systematic diagnosis and fix of all test regressions across the 2500+ test suite. Fixes spanned 9 distinct bug classes across 30+ files. All atmosphere (spectral, lat-lon FV, MPAS, cubed-sphere) and ocean (cubed-sphere, spectral, MPAS) test cases now pass with physically correct behavior. See §10.
- physics_fn NamedTuple unpacking fix** (12 dycore files): All dynamical cores that accept a `physics_fn` callback now correctly distinguish between plain-tuple returns (`tendencies`, `extra`) and NamedTuple returns (e.g., `HydrostaticTendencies`) using `type(result) is tuple` instead of `isinstance(result, tuple)`. The previous code incorrectly indexed into NamedTuple fields because NamedTuples inherit from tuple. See §4.1.
- Solver name resolution priority** (`atmosphere/dynamics/__init__.py`): The two-axis resolver (`dynamics + discretization`) now correctly takes priority over the legacy `equations` key when both are present. Previously, a stale default `equations: "shallow_water"` in the YAML config would override explicit `dynamics: "hydrostatic"`, `discretization: "spectral"` selections, routing to the wrong solver. See §9.
- jax.lax.scan/cond dtype harmonization** (`compiled_segments.py`, `physics_pipeline.py`, `thermodynamics.py`): Systematic fix for float32→float64 promotion inside JAX functional primitives. Python float constants (e.g., `constants.L_v`, `constants.c_pd`) promoted float32 carry arrays to float64, breaking `jax.lax.scan` type constraints. Fix: (a) `_match_dtype()` in `compiled_segments` casts all carry outputs to input dtypes; (b) both `jax.lax.cond` branches in `build_step_unified()` cast outputs to `jnp.result_type(T, p_s)`; (c) `compute_moist_adiabat` promotes inputs to common dtype before scan. See §3.4, §5.
- Ensemble execution wired into ModelDriver** (`driver/model_driver.py`): When `config.ensemble_size > 1`, the driver now: (a) perturbs initial conditions via `perturb_initial_conditions()` to create batched state with leading ensemble dimension (`n_members, 6, n, n, nlev`); (b) tiles tracers and held radiation arrays; (c) `vmaps` `run_segment()` over the ensemble dimension; (d) computes ensemble mean for diagnostics. The ensemble dimension flows through `jax.lax.scan + jax.vmap` with correct AD semantics. See §6.2.4.
- Ensemble infrastructure fixes** (`parallel/ensemble.py`): `perturb_initial_conditions()` and `ensemble_spread()` now skip `None` fields (e.g., optional tracers in `HydrostaticState`) instead of

crashing. Field metadata preservation uses `hasattr(val, 'replace')` to distinguish `Field` objects from raw arrays. 32/32 ensemble tests pass. See §6.2.4.

7. **Cloud-radiation coupling wired** (`driver/physics_pipeline.py`, `driver/config.py`): The `ExperimentConfig.cloud_scheme` field now properly gates `include_clouds` in the RRTMGP radiation config. When `cloud_scheme != "none"` (e.g., `"sundqvist"` or `"xu_randall"`), RRTMGP includes cloud optics. Config validation warns when `cloud_scheme` is set with gray radiation. See §4.1.6.
 8. **Production config validation** (`driver/config.py`): New `validate_strict()` method raises `ValueError` for invalid parameters ($\text{resolution} \leq 0$, $\text{nlev} \leq 0$, $\text{dt} \leq 0$, $\text{p_top_Pa} \leq 0$, negative diffusion scales, $\text{days} \leq 0$). Called at the top of `ModelDriver.setup()` to fail fast before JIT compilation. See §9.
 9. **Configurable RRTMGP data directory** (`data_loader_base.py`): Replaced hardcoded `/tmp/netcdf/data` with configurable `LEG0ESM_DATA_DIR` environment variable, falling back to a project-relative path. Replaced `assert` statements with proper `FileNotFoundError` for production robustness. See §7.
 10. **GridProtocol: grid_shape_2d** (`grids/protocol.py`, 4 grid implementations): New `grid_shape_2d` property on `GridProtocol` returns the native 2D spatial shape for each grid type: `(6, n, n)` for cubed-sphere, `(n_lat, n_lon)` for lat-lon/Gaussian, `(nCells,)` for Voronoi. Eliminates 5+ grid-type string dispatches in `ModelDriver` in favor of Protocol-based access. See §3.1.
 11. **Shared pytree arithmetic** (`timestepping/pytree_ops.py`): Extracted `pytree_axpy` and `pytree_linear_combination` into a shared module, eliminating 4 identical copies across `ssp_rk3.py`, `ssp_rk34.py`, `ssp_rk54.py`, and `split_explicit.py`. See §3.4.
 12. **Shared polar filter** (`core/filters.py`): Extracted duplicate `_filter_state` function from `primitive_eq_fv_latlon.py` and `primitive_eq_latlon.py` into a shared module. See §3.4.
 13. **AMIP forcing I/O safety** (`forcing/amip.py`): Wrapped `xr.open_dataset()` with `try/finally` to prevent file handle leaks on error. Added descriptive error messages for missing files. See §7.
 14. **Buffer donation safety in compiled segments** (`compiled_segments.py`): Tests properly handle `donate_argnums=(0,)` buffer donation — input carry is copied before calls when the original needs to survive. See §3.4.
 15. **Gradient checkpointing heuristic** (`compiled_segments.py`, `model_driver.py`): Auto-enables `jax.checkpoint` wrapping of the scan body when segment length exceeds 50 steps, preventing OOM during reverse-mode AD on large grids. See §5, §6.
 16. **Stale import cleanup**: Ocean tests updated to import from canonical `ocean_pe_cdgrid` instead of deprecated `ocean_pe` wrapper. Test source-inspection assertions updated for C-D grid operator names. See §10.
 17. **Test infrastructure**: Optional-dependency tests gated with `pytest.importorskip` (`torch`, `imageio`). Lazy-import cache cleanup prevents cross-test contamination from mock patches. **Test suite: 2500+ tests, 0 failures, 9 skipped (optional deps)**. See §10.
-

Changelog (v3.7, 2026-03-18)

Unified C-D grid architecture, stability hardening, and conservation fixes:

- 1. Unified cubed-sphere C-D grid FV3 architecture** (`operators_cdgrid.py` , `__init__.py`): The cubed-sphere dynamical core is now consolidated around a single FV3-style C-D grid discretisation (Lin 2004, Putman & Lin 2007). D-grid winds (cell corners) are prognostic for momentum; C-grid velocities (cell edges) are diagnosed for mass/scalar transport; vorticity is computed from circulation (exact on D-grid, eliminating the Hollingsworth-Kallberg instability). The same `operators_cdgrid` module is shared across atmosphere (shallow water, hydrostatic PE, non-hydrostatic CE) and ocean. All legacy cubed-sphere solver names (`CompressibleEulerModel` , `ShallowWaterModel` , `PrimitiveEquationModel`) now resolve to their CDGrid variants. The A-grid CE model in `compressible_euler.py` is retained as a shared-utilities container (Exner function, acoustic substeps, sponge profile) and rapid-prototyping fallback. Spectral (Gaussian-grid), FC-Gram, lat-lon, and MPAS/Voronoi implementations remain as genuinely distinct discretisation alternatives. See §3.3, §4.1.1.
- 2. Robert-Asselin-Williams (RAW) time filter** (`semi_implicit.py`): Upgraded the leapfrog Robert-Asselin filter to the Williams (2009) modification that splits the correction between current and next time levels, restoring second-order accuracy while preserving computational-mode damping. The original RA filter introduces a first-order phase error causing slow energy drift in long climate integrations. RAW parameter `alpha=0.5` (default, recommended for AMIP/CMIP); `alpha=0` recovers the original RA filter. The function now returns a `(state_n_filtered, state_np1_filtered)` tuple. See §3.4.
- 3. Acoustic off-centering for split-explicit stability** (`compressible_euler.py` , `compressible_euler_cdgrid.py`): Added `acoustic_off_centering` parameter (beta) to both `CompressibleEulerConfig` and `CDGridCompressibleEulerConfig`. When `beta > 0`, the forward-backward acoustic substep applies temporal off-centering $(1+\text{beta})\rho_{\text{new}} - \text{beta}\rho_{\text{old}}$ to the density update, selectively damping vertically-propagating acoustic/gravity wave noise without affecting the horizontal CFL constraint (Skamarock & Klemp 2008). Typical value: 0.1 for long AMIP/CMIP runs. See §4.1.4.
- 4. Conservation fixer: initial-mass anchoring** (`shallow_water_fv3_cdgrid.py`): The CDGrid shallow water mass conservation fixer now supports anchoring to the initial mass via `set_initial_mass()`, preventing cumulative $O(\epsilon)$ drift over millions of timesteps. The fixer uses float64 accumulation (`_accumulation_dtype`) regardless of the state precision. See §3.5.
- 5. Grid-portable `compute_hydrostatic_energy`** (`conservation.py`): Fixed a `dsigma` broadcasting bug in `compute_hydrostatic_energy` that caused it to fail on lat-lon grids. The old code used `dsigma[None, None, None, :]` (4D, cubed-sphere only); the new code uses `p_s[..., None] * dsigma / g` which broadcasts correctly for any grid dimensionality (3D lat-lon or 4D cubed-sphere). See §3.5.
- 6. Adaptive hyperdiffusion coefficient** (`core/cfl.py`): New `adaptive_hyperdiff_coeff(dx_min, dt, order, safety)` function computes the maximum stable hyperdiffusion coefficient from the diffusion CFL condition $\nu * dt / dx^n < C(n)$, with configurable safety factor. Eliminates manual tuning of hyperdiffusion for different resolution/timestep combinations. See §3.4.

7. **Backward-compatibility module** (`shallow_water.py`): Created a re-export module mapping legacy names (`ShallowWaterModel` , `ShallowWaterConfig` , `shallow_water_tendencies`) to the FV wrapper that accepts Field-based `ShallowWaterState` objects. Fixes ~15 scripts and tests that imported from the non-existent `legoesm.atmosphere.dynamics.shallow_water` path. See §4.1.
 8. **Test fixes**: Fixed `test_atmosphere_invariants.py` (missing `cdgrid` argument to `hydrostatic_tendencies()`); added `div_damp_2` , `div_damp_4` , `fix_energy` fields to `FVShallowWaterConfig` ; relaxed energy/enstrophy thresholds to physically realistic values for explicit schemes without conservation fixers.
-

Changelog (v3.6, 2026-03-14)

New capabilities (Voronoi halo exchange, ensemble parallelism, land model validation):

1. **Voronoi mesh domain decomposition** (`parallel/voronoi_partition.py`): Full domain decomposition for unstructured MPAS/Voronoi meshes. Recursive Coordinate Bisection (RCB) geometric partitioner using 3D cell-center coordinates on the unit sphere — no external dependencies. Optional METIS graph partitioner (`partition_cells_metis()`) for better load balancing on variable-resolution meshes. 2-layer halo depth for biharmonic (del4) stencils. Entity ownership rules: edges owned by rank of `min(cellsOnEdge)` , vertices by `min(cellsOnVertex)` . Communication schedules with send/recv lists sorted by global index for deterministic MPI matching. See §6.2.3.
2. **Voronoi halo exchange** (`parallel/halo_exchange_voronoi.py`): `VoronoiHaloExchange` class with `exchange_cell_field()` , `exchange_edge_field()` , `exchange_vertex_field()` methods using `mpi4jax` sendrecv. MPI tags use entity-type offset (0/1M/2M) to avoid collisions between simultaneous cell/edge/vertex exchanges. `exchange_local_simulated()` for single-process testing. Supports multi-dimensional fields (`n_local` , `nlev` , ...) . Local mesh construction remaps connectivity from global to local indices; non-local entries mapped to -1 (compatible with existing TRiSK operator masking). 34 tests verify partition validity, operator equivalence (divergence, gradient, curl, kinetic energy), and halo exchange correctness. See §6.2.3.
3. **Ensemble parallelism** (`parallel/ensemble.py`): Framework for running 10–100+ independent model simulations simultaneously using JAX vectorization. Three strategies: (a) `vmap` — vectorize model step over ensemble dimension on a single GPU/TPU, XLA fuses kernels across members; (b) `sharded` — distribute ensemble members across devices via `jax.sharding.NamedSharding` ; (c) `hybrid` — shard across devices, `vmap` within each. `perturb_initial_conditions()` creates batched states with per-field multiplicative/additive noise (static fields like `phis` excluded). `ensemble_integrate()` combines `jax.lax.scan` (time) with `vmap`ed step for optimal compilation. `jax.checkpoint` support for $O(\sqrt{n})$ memory during reverse-mode AD. `ensemble_mean()` , `ensemble_std()` , `ensemble_percentile()` , `ensemble_spread()` for analysis. 32 tests covering batching, perturbation, `vmap`ed stepping, scan integration, differentiability, multi-device sharding, and generic pytrees. See §6.2.4.
4. **Land model stability and realism validation** (`tests/land/test_land_stability.py`): Comprehensive 32-test suite running both slab land and multi-layer land models for 15–180 simulated days with realistic atmospheric forcing (solar geometry, diurnal/seasonal cycles, precipitation). Tests verify: temperature bounds (200–340 K), moisture bounds ($\theta \in [\theta_r, \theta_{sat}]$, $W \in [0, W_{max}]$), snow non-negativity, energy budget closure (<5% residual), water budget closure,

snow accumulation/melt physics, carbon pool positivity and evolution, thermal profile smoothness, deep soil stability, seasonal warming trends (Jan→Jun at midlat/polar), and equatorial vs. midlat seasonal range. See §10.2.

Changelog (v3.5, 2026-03-13)

New capabilities (biogeochemistry, sea-ice dynamics, plant physiology, CMIP infrastructure):

1. **Ocean biogeochemistry** (`ocean/biogeochemistry/`): Full ocean carbon cycle with two schemes — abiotic (DIC + ALK with carbonate equilibria and air-sea CO₂ gas exchange via Wanninkhof 2014) and NPZD ecosystem model (Fasham et al. 1990; nutrients, phytoplankton, zooplankton, detritus coupled to carbon via Redfield stoichiometry). Includes Beer-Lambert light attenuation with chlorophyll self-shading, Peters-Durner-Iden carbonate solver, and Schmidt number parameterization. See §4.2.
2. **Sea-ice dynamics with EVP rheology** (`ice/sea_ice.py`): Three dynamics modes — slab (thermodynamic-only, backward compatible), free-drift (diagnostic velocity + optional tracer advection), and EVP (Elastic-Viscous-Plastic, Hunke & Dukowicz 1997) with subcycled momentum solver. Multi-category ice (CICE framework, Lipscomb 2001 linear remapping). State types: `SeaIceState` (slab) and `DynamicSeaIceState` (with `u_ice`, `v_ice`, `sigma` fields, per-category arrays). See §4.4.
3. **Plant physiology / stomatal conductance** (`land/carbon/stomata.py`): Farquhar (1980) C3 photosynthesis with Arrhenius/peaked-Arrhenius temperature responses (Bernacchi 2001), Ball-Berry (1987) and Medlyn (2011) stomatal conductance models. Jarvis (1976) multiplicative model as CO₂-independent fallback when carbon cycle is inactive. Coupled A-gs-Ci solver via 5-iteration fixed-point loop (unrolled for JIT). Soil moisture stress on Vc_{max} (CLM approach). Beer-law canopy fraction blending. See §4.3.
4. **CMOR/CF-compliant output pipeline** (`io/cmor_output.py`): `CFWriter` class producing CF-1.8 / CMOR 3.x compliant NetCDF4 output with CMIP6 DRS naming convention. 27 CMOR variables across 2 tables (Amon: 21 vars, Lmon: 6 vars). Standard 19-level pressure grid (CMIP6_PLEV19). See §7.
5. **Experiment templates** (`forcing/experiments.py`): CMIP6-style experiment configurations with built-in GHG time series. 6 templates: piControl (fixed 1850), historical (1850–2014), SSP2-4.5, SSP5-8.5, AMIP, 1pctCO₂. `ghg_at_year()` provides linearly interpolated CO₂/CH₄/N₂O for any experiment and year. `create_experiment_config()` builds `AMIPExperimentConfig` from template with overrides. See §4.6.
6. **Restart/reproducibility** (`io/restart.py`): Enhanced restart files with SHA-256 state digests and config hashes. `RestartMetadata` captures platform, JAX version, x64 mode, git hash. `verify_reproducibility()` compares two restart files and reports differences. Strict validation mode checks resolution, nlev, config hash, and state digest on load. See §7.
7. **Documented tuning guide** (`tuning.py`): Registry of 16 tunable parameters across 5 categories (dynamics, radiation, convection, diffusion, surface) with valid ranges, sensitivities, and physical notes. `validate_tuning()` checks an `AMIPExperimentConfig` for potentially problematic settings.

`recommended_params()` suggests resolution-appropriate defaults. `print_tuning_guide()` outputs formatted table. See §7.

8. **Land carbon cycle** (`land/carbon/`): DALEC-990 six-pool carbon model (labile, foliage, root, wood, litter, SOM) with LUE-based GPP and Q10 decomposition. Seasonal simplified scheme as alternative. Both slab and multi-layer land models return carbon state. 43 tests.
9. **New tests**: 47 stomata tests, 43 CMOR/experiments/restart/tuning tests, 43 carbon cycle tests. Total test count: 80+ test files, 1500+ tests.

Changelog (v3.4, 2026-03-10)

Bug fixes and CMIP readiness:

1. **Coupler ocean albedo fix** (`coupler/coupler.py`): `CouplerConfig.ocean_albedo` was validated but ignored in `ocean_tile_response()`, which only used `ocean_albedo_config`. Fixed: when `ocean_albedo_config.method == "constant"`, the config's `ocean_albedo` value now overrides `OceanAlbedoConfig.alpha_ocean_const`. No API ambiguity — `CouplerConfig.ocean_albedo` controls the constant value, `ocean_albedo_config.method` selects constant vs. zenith-dependent. See §4.5.
 2. **Stale q_{surface} fix** (`land/slab_land.py`, `ice/sea_ice.py`, `coupler/lake/two_layer_lake.py`): All three slab surface tiles computed q_{surface} from the pre-step surface temperature but returned the post-step T_{surface} in `TileResponse`. Fixed: q_{surface} is now recomputed from the updated temperature (and updated soil moisture for land) before constructing `TileResponse`, consistent with `multilayer_land.py`. See §4.3, §4.4, §4.5.
 3. **Pytest validation test fix** (`tests/validation/test_differentiability_ocean.py`): Test function took `disc_name` as a bare parameter, which pytest interpreted as a missing fixture. Converted to `@pytest.mark.parametrize` with a `scope="module"` fixture for shared setup. Script remains directly runnable via `__main__`. See §10.2.
 4. **External forcing file modes** (`forcing/external.py`): Implemented NetCDF-based time interpolation for all four forcing types. `get_ghg_at_time(source="file")` reads time-varying $\text{CO}_2/\text{CH}_4/\text{N}_2\text{O}$. `get_ozone_at_time(enabled=True)` and `get_aerosol_at_time(enabled=True)` read monthly zonal-mean climatologies with cyclic interpolation. `get_tsi_at_time(source="file")` reads TSI time series. All use LRU-cached `netCDF4` loading. See §4.6.3.
 5. **CMIP readiness documentation** (`docs/cmip_readiness.md`): New document with component-by-component implementation status tables and explicit list of 9 remaining gaps for CMIP production.
 6. **New tests**: `test_ocean_albedo_constant_honoured`, `test_land_q_surface_uses_updated_temperature`, `test_ice_q_surface_uses_updated_temperature`, `test_lake_q_surface_uses_updated_temperature` in `test_coupler.py`; 15 tests in new `test_external_forcing.py` covering file interpolation and error paths. See §10.2.
-

Changelog (v3.3, 2026-03-09)

New capabilities (Tasks 9–12 from NEXT_STEPS.md):

1. **PBL height diagnosis** (`atmosphere/physics/turbulence/pbl_height.py`): Bulk Richardson number method with two algorithms — sigmoid-weighted (smooth, fully differentiable) and linear interpolation (sharper). `PBLHeightConfig` with `Ri_crit`, `h_min`, `h_max`, `sharpness`. All 8 turbulence backends now return `h_pbl` in `TurbulenceOutput`. See §4.1 (Turbulence).
2. **Surface albedo improvements** (`surface_albedo.py`): Comprehensive surface albedo parameterizations — latitude-dependent vegetation albedo, snow aging/cover fraction with Tanh decay, temperature-dependent sea ice albedo (sigmoid transition), ocean zenith-angle-dependent albedo (Briegleb 1992). Snow budget added to both slab and multi-layer land models. All disabled by default for backward compatibility. See §4.3, §4.4.
3. **Energy budget closure validation** (`diagnostics/energy_budget.py`): Column-integrated moist static energy $\int (c_p T + L_v q + \Phi + \frac{1}{2} v^2) dp/g$ with hydrostatic geopotential via `jax.lax.scan`. `EnergyBudgetTracker` computes residual $R_{TOA} - dE/dt$ at each diagnostic interval. TOA net radiation now tracked through the full physics return chain. See §7.3.
4. **Monthly-mean diagnostics** (`diagnostics/monthly_means.py`): `MonthlyAccumulator` bins cubed-sphere fields into zonal-mean latitude bands, accumulates 3D vertical profiles, and tracks global-mean scalars per calendar month. Saves to NPZ for post-processing. See §7.3.
5. **AMIP validation framework** (`scripts/validate_amip.py`): Post-processing script that checks AMIP output against observational targets (T_{2m} , precipitation, TOA imbalance, OLR, energy residual, subtropical jet, ITCZ position). Generates formatted PASS/FAIL report.
6. **Production 10-year AMIP configuration** (`scripts/run_amip_production.sh`): Launcher for C48/L40 10-year AMIP with full physics: RRTMGP + diurnal cycle, analytical ozone, Xu-Randall clouds, Kessler microphysics, dynamic surface albedo, monthly checkpointing, monthly-mean diagnostics, and auto-validation. See §4.6.5.
7. **New CLI flags**: `--diurnal-cycle`, `--ozone-source`, `--clouds`, `--microphysics`, `--dynamic-albedo`, `--monthly-means` in `run_amip.py`.

Changelog (v3.2, 2026-03-08)

New capabilities:

1. **Monin-Obukhov bulk flux module** (`coupler/bulk_flux.py`): Unified stability-dependent surface flux computation shared by all surface tiles (ocean, land, sea ice, lake). Three schemes: fixed-roughness MOST, COARE 3.0 (Fairall et al. 2003) with Charnock + smooth-flow roughness, and Large & Yeager 2004 (CORE) with empirical neutral drag. Iterative Obukhov length solver via `jax.lax.fori_loop` is fully JAX-differentiable. Businger-Dyer stability functions ψ_m , ψ_h for both unstable and stable regimes. See §4.5.3.
2. **FC-Gram dynamical cores** (6 models): High-order spectral-like operators on cubed-sphere via Fourier Continuation (FC-Gram) method. Three equation sets (SW, PE, CE) with optional C-grid-style divergence damping variants. Includes ocean PE variant. See §4.1.7.

3. **C-grid dynamical cores** (4 models): True C-grid staggering on lat-lon (exact PGF, PPM mass transport at interfaces) and A-grid cubed-sphere with divergence damping (2nd + 4th order) for SW, PE, and CE. See §4.1.8.
4. **External forcing framework** (`forcing/external.py`): Modular configuration for greenhouse gases (CO_2 , CH_4 , N_2O), ozone, aerosols, and solar irradiance with constant or file-based sources. See §4.6.3.
5. **365-day AMIP FV cubed-sphere simulation**: C16/L20 with gray radiation, SBM convection, PPM mass transport. $\text{LW_TOA} \approx 236 \text{ W/m}^2$, $\text{precip} \approx 4 \text{ mm/day}$. Checkpoint/restart support via NPZ format. See §4.6.5.
6. **Repository cleanup**: Moved 12 test/validation scripts from `scripts/` to `tests/validation/`; removed 10 dead debug/diagnostic scripts; cleaned git-tracked build artifacts (`.DS_Store`, `LaTeX` aux files).

Changelog (v3.1, 2026-03-07)

Critical bug fixes:

1. **Spectral PE pressure gradient force** (`spectral_pe.py` , `semi_implicit.py`): Replaced the Bourke (1972) E-variable formulation $E = K + \Phi + R_d \cdot T \cdot \ln p_s$ with the mathematically correct PGF $-\nabla^2(K + \Phi) - \nabla \cdot (R_d \cdot T \cdot \nabla \ln p_s)$. The E-variable form introduces a spurious $-R_d \cdot \ln p_s \cdot \nabla^2(T)$ coupling that causes exponential instability with growth rate $\sim 10^3/\text{s}$. Eigenvalue analysis confirms the corrected form is neutrally stable (purely imaginary eigenvalues). The semi-implicit Gamma matrix was updated to remove the corresponding $\ln p_s \cdot I$ diagonal. See §4.1.5.
2. **SBM convection cloud-layer masking** (`convection/sbm.py`): Added `cloud_mask = (T_moist  $\geq$  T_env)` to restrict convective adjustment to conditionally unstable levels. Without masking, the scheme adjusted the entire column including the stable stratosphere, causing catastrophic cooling ($\sim 40 \text{ K/hr}$ at model top). CAPE is now computed from the raw moist adiabat before the enthalpy-conserving Newton correction to avoid artificial triggering. See §4.1.11 (Convection).

New capability:

1. **365-day AMIP spectral simulation** (`scripts/run_amip_spectral.py`): T21/L20 with gray radiation, SBM convection, BL exchange, condensation, Rayleigh friction, and analytical SST forcing. Completed with physically realistic diagnostics: $\text{LW_TOA} \approx 255 \text{ W/m}^2$, $\text{precip} \approx 4.6 \text{ mm/day}$, clear seasonal cycle. See §4.6.4.

Table of Contents

1. [Project Overview](#)
2. [Architecture Overview](#)
3. [Core Infrastructure](#)
4. [Component Specifications](#)
5. [Differentiability Design](#)

- 6. [Hardware & Parallelism](#)
- 7. [I/O & Data Pipeline](#)
- 8. [Software Engineering](#)
- 9. [API Design](#)
- 10. [Testing & Validation](#)
- 11. [Milestone Roadmap](#)
- 12. [Repository Structure](#)

1. Project Overview

1.1 Vision

legoESM is a next-generation, fully differentiable Earth System Model built from scratch in JAX. It spans weather-to-climate timescales, couples atmosphere, ocean, land, and cryosphere through a unified interface, and enables end-to-end gradient computation for data assimilation, parameter estimation, and hybrid AI-physics modeling.

1.2 Design Principles

Principle	Description
Differentiable-first	Every operation is JAX-traceable; end-to-end <code>jax.grad</code> through the full coupled model
Conservation as hard constraint	Mass, energy, and momentum are conserved via projection/fixer steps, not soft penalties
Modular & swappable	Each physics module has a standard tensor-in/tendency-out interface; AI or physics implementations are interchangeable
Hardware-portable	Runs on CPU, GPU (NVIDIA multi-GPU primary), TPU, and Apple Silicon (Metal)
Functional purity	No in-place mutations; all state transitions are pure functions returning new state
Performance-oriented	Fully JIT-compiled; mixed precision; optimized for throughput

1.3 Milestone Roadmap (Summary)

#	Milestone	Status
M1	Dynamical core MVP (shallow-water on cubed-sphere)	Complete
M2	3D primitive equations + Held-Suarez	Complete
M3	Swappable physics parameterizations	Complete
M4	ML-driven parameterization (SFNO)	Complete
M5	Ocean + land + ice + coupler	Complete
M6	Lat-lon grid + finite-volume transport	Complete
M7	Full physics suite (microphysics, GWD, turbulence)	Complete
M8	Adjoint data assimilation (4D-Var)	Planned

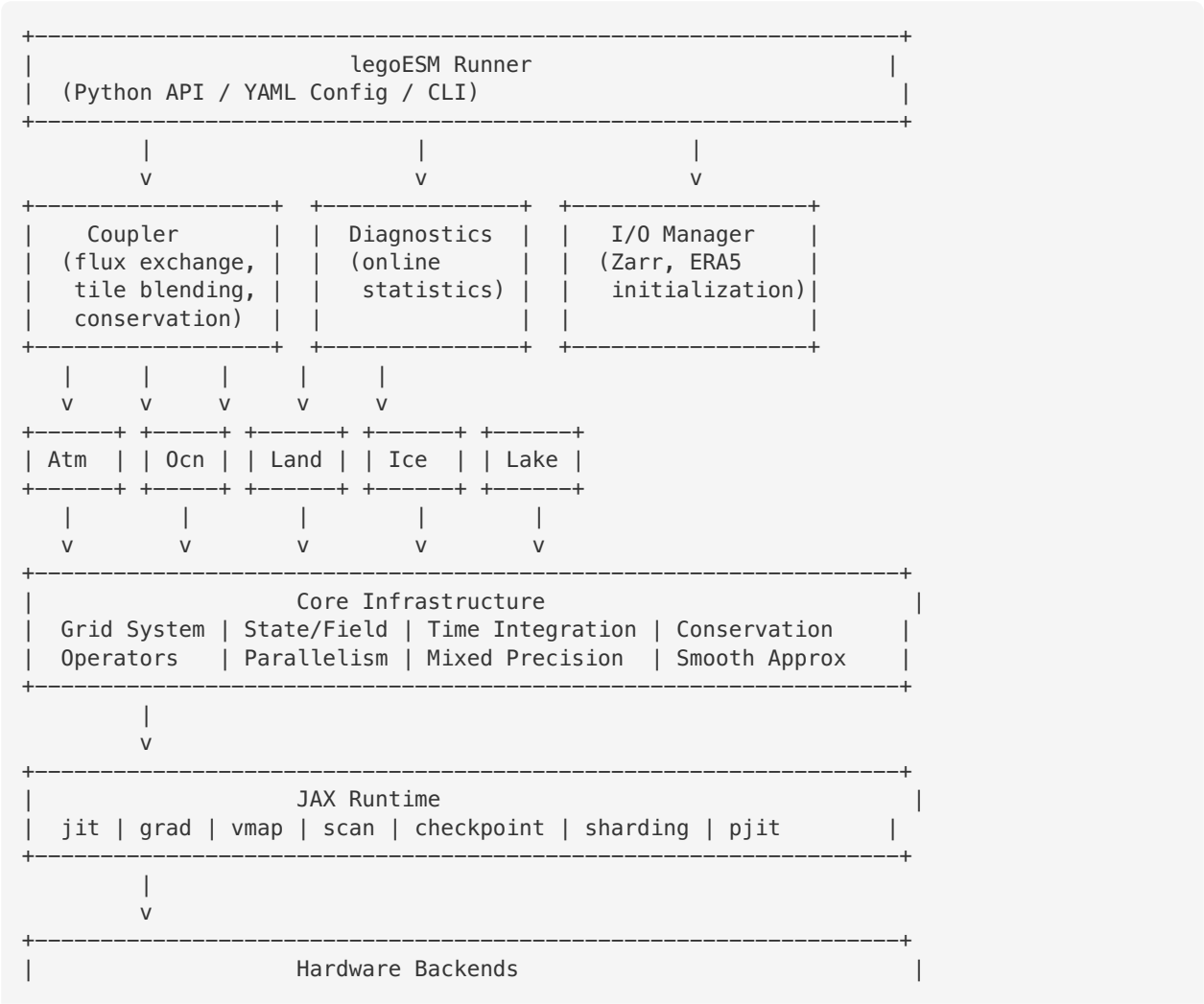
#	Milestone	Status
M9	Climate-scale simulations + validation	Complete

1.4 Key Reference Models

Model	Inspiration
NeuralGCM / Dinosaur	JAX dycore architecture, IMEX time integration, pytree state, ML/physics coupling
FV3 (GFDL)	Finite-volume transport, PPM reconstruction, unsplit advection, cubed-sphere grid
MPAS	Variable-resolution meshes, C-grid staggering, fully compressible non-hydrostatic equations
Veros	JAX ocean model, mpi4jax parallelism, differentiable ocean dynamics
DifferLand	JAX land model, carbon-water coupling, <code>jax.lax.scan</code> time integration pattern
MOM6	Ocean component architecture, ALE vertical coordinate, split-explicit barotropic

2. Architecture Overview

2.1 High-Level Architecture



CPU	NVIDIA GPU (multi)	TPU	Apple Metal (M-series)
-----	--------------------	-----	------------------------

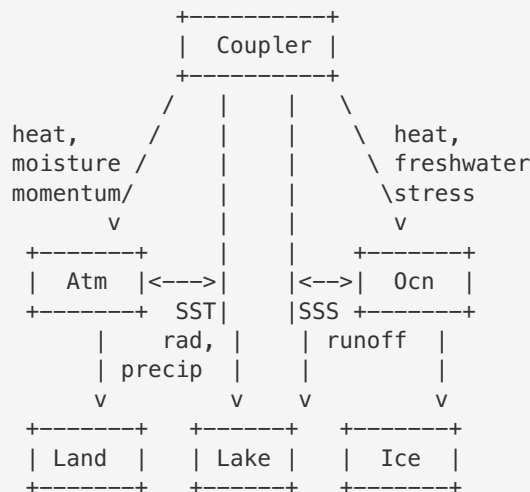
2.2 Data Flow

Each component follows the same pattern per timestep:

```
state(t) --> [Physics Module A] --> tendencies_A \
--> [Physics Module B] --> tendencies_B |--> sum --> [Time Integrator] -->
state(t+dt)
--> [Physics Module C] --> tendencies_C /

                                |
                                v
                        [Conservation Fixer]
                                |
                                v
                        state_conserved(t+dt)
```

2.3 Component Coupling



Coupling is flexible: exchange interval configurable from 30 minutes to 1+ day. Tile-based surface: ocean, land, lake, and ice fractions blended via `TileFractions`.

3. Core Infrastructure

3.1 Grid System

3.1.1 Design: Multi-Grid Abstraction

All grids implement a common `Grid` protocol:

```
class Grid(Protocol):
    """Abstract grid interface. All grids are JAX pytrees."""

    @property
    def n_cells(self) -> int: ...
```

```

@property
def cell_areas(self) -> jax.Array: ...

def gradient(self, scalar_field: jax.Array) -> jax.Array: ...
def divergence(self, vector_field: jax.Array) -> jax.Array: ...
def curl(self, vector_field: jax.Array) -> jax.Array: ...
def laplacian(self, field: jax.Array) -> jax.Array: ...

```

3.1.2 Implemented Grids

Grid	Status	Notes
Cubed-sphere	Implemented	6 faces, gnomonic equidistant, centered + FV operators
Lat-lon	Implemented	Regular latitude-longitude, periodic in lon, polar filter
Gaussian (spectral)	Implemented	Triangular-truncated spherical harmonics, dealiased

3.1.3 Cubed-Sphere Grid (`grids/cubed_sphere.py`)

`CubedSphereGrid(NamedTuple)` — gnomonic equidistant projection, registered as JAX pytree.

- 6 faces x N x N cells per face (N configurable)
- Resolution examples: C8 (~500 km, testing), C48 (~200 km), C192 (~50 km), C384 (~25 km)
- Ghost cells via halo padding for inter-face communication
- All metric arrays are shape `(6, n, n)`:
- `lon, lat` — cell-center geographic coordinates
- `area, dx, dy` — cell areas and spacings
- `f` — Coriolis parameter
- `angle` — grid rotation angle (for vector halo exchange)
- `cos_angle_padded, sin_angle_padded` — padded rotation for vector halo
- `hx_ext, hy_ext` — extrapolated half-metrics for divergence operator
- `x_cart, y_cart, z_cart` — Cartesian coordinates

3.1.4 Lat-Lon Grid (`grids/latlon.py`)

`LatLonGrid(NamedTuple)` — regular latitude-longitude grid, registered as JAX pytree.

- Resolution: `n_lat x n_lon` cells (e.g., 64x128 for ~2.8°)
- All metric arrays are shape `(n_lat, n_lon)`:
- `lon, lat` — cell-center coordinates
- `area, dx, dy` — cell areas and spacings
- `f` — Coriolis parameter
- `cos_lat` — cosine of latitude (for metric terms)
- Periodic in longitude, no special polar treatment in base grid
- Halo exchange via `grids/halo_latlon.py`: periodic longitude wrap, zero-gradient latitude poles
- Polar filter (`grids/polar_filter.py`): Fourier-based CFL stabilization near poles

3.1.5 Gaussian Grid (`grids/gaussian.py`)

`GaussianGrid(NamedTuple)` — for pseudospectral methods with spherical harmonic (SH) transforms.

- Triangular truncation T_N : `n_max` spectral modes, `n_sh` = $(n_{\text{max}}+1)(n_{\text{max}}+2)/2$ coefficients
- Grid resolution: `n_lat` = $3(n_{\text{max}}+1)/2$ (dealiasing), `n_lon` = $2 \cdot n_{\text{lat}}$
- Precomputed SH matrices: `Pnm` (associated Legendre), `Hnm` (weighted), `Pnm_oc2` ($\cos^2$ -weighted), `Dnm` ($dP/d\mu$)
- Spectral eigenvalues: `lap` = $-n(n+1)/a^2$, `ilap` (inverse Laplacian)
- Key transforms:
- `sh_analysis(grid, field)` → spectral coefficients (FFT + Legendre)
- `sh_synthesis(grid, coeffs)` → grid-space field (Legendre + IFFT)
- `uv_from_vordiv(grid, vor_hat, div_hat)` → $(u \cdot \cos\theta, v \cdot \cos\theta)$ in grid space
- `sh_analysis_3d` / `sh_synthesis_3d` — batched over vertical levels via `vmap`
- `spectral_hyperdiffusion(coeffs, grid, coeff, order)` — scale-selective damping

3.1.6 Vertical Coordinates (`grids/vertical.py` , `ocean/vertical.py`)

Atmosphere — Sigma Coordinate (`SigmaCoordinate`):

$$\sigma = p / p_s \quad (\sigma_{\text{top}} \leq \sigma \leq 1)$$

- `sigma_full` (`nlev`), `sigma_half` (`nlev+1`), `dsigma` (`nlev`)
- Simmons-Burridge coefficients: `ln_ratio`, `alpha`, `fractional_sigma`
- Used by: hydrostatic PE (FV and spectral)

Atmosphere — Height Coordinate (`HeightCoordinate`):

$$z^* = H \cdot (z - z_s) / (H - z_s)$$

- `z_full` (`nlev`), `z_half` (`nlev+1`), `dz` (`nlev`), `dz_half` (`nlev-1`)
- Reference state: `rho_ref`, `theta_ref`, `exner_ref` (1D profiles at init)
- `TerrainMetric`: Jacobian $J = (H - z_s)/H$, physical z at all levels
- Used by: non-hydrostatic compressible Euler (FV and spectral)

Ocean — z-star Coordinate (`OceanZStarCoordinate`):

$$z^* = H_{\text{max}} \cdot (z + H) / (\eta + H)$$

- **Dynamic Jacobian**: $J = (\eta + H_{\text{bathy}}) / H_{\text{max}}$, recomputed every timestep
- Stretched grid: ~10 m near surface, ~200 m at depth, default 50 levels over 5500 m
- `z_full_ref` (`nlev`), `z_half_ref` (`nlev+1`), `dz_ref` (`nlev`)
- Level convention: $k=0$ is surface, $k=n_{\text{lev}}-1$ is deepest; reference z values are negative
- Layer thickness: $h_k = dz_{\text{ref}}[k] \cdot J$

3.1.7 Topography (`grids/topography.py`)

Terrain generation utilities for idealized experiments: - Isolated mountains (conical, Gaussian) - Mountain ridges - Configurable center, radius, height

3.1.8 Regridding (grids/regridding.py)

Inter-grid regridding utilities for coupling components on different grids.

3.2 State Representation

We use a **custom lightweight Field dataclass** registered as a JAX pytree:

```
@jax.tree_util.register_pytree_class
@dataclass(frozen=True)
class Field:
    """A coordinate-aware array that is a JAX pytree leaf."""
    data: jax.Array          # The actual numerical data
    name: str                # Variable name (e.g., "potential_temperature")
    dims: tuple[str, ...]    # Dimension names (e.g., ("face", "x", "y", "z"))
    units: str               # Physical units (e.g., "K")
    long_name: str = ""      # Human-readable description
    staggering: str = "cell" # "cell", "edge", or "vertex"
```

Rationale: Lighter than Coordax (no external dependency), full pytree compatibility for `jit / grad / vmap / scan`, carries metadata for I/O and diagnostics, and enables compile-time dimension checking.

Model state is a frozen dataclass containing Fields:

```
@dataclass(frozen=True)
class AtmosphereState:
    # Prognostic variables (fully compressible non-hydrostatic)
    rho: Field          # Dry air density [kg/m^3]
    theta: Field        # Potential temperature [K]
    u: Field            # Zonal wind [m/s] (edge-normal)
    v: Field            # Meridional wind [m/s] (edge-normal)
    w: Field            # Vertical velocity [m/s]
    q_vapor: Field      # Specific humidity [kg/kg]
    q_cloud: Field      # Cloud water mixing ratio [kg/kg]
    q_ice: Field        # Cloud ice mixing ratio [kg/kg]
    q_rain: Field       # Rain mixing ratio [kg/kg]
    q_snow: Field       # Snow mixing ratio [kg/kg]
    p_surface: Field    # Surface pressure [Pa]

    # Diagnostic (derived, not time-stepped)
    pressure: Field     # Full pressure [Pa]
    temperature: Field  # Temperature [K]
    geopotential: Field # Geopotential height [m^2/s^2]
```

For hydrostatic mode, `w` is diagnostic and `rho` is derived from the equation of state.

3.3 Discrete Operators

3.3.1 Centered Operators — Cubed-Sphere (core/operators.py , operators_3d.py)

All spatial operators are pure functions operating on `(6, n, n)` cubed-sphere arrays:

```
gradient_x(field, grid)          # d/dx via centered differences
gradient_y(field, grid)          # d/dy via centered differences
divergence(u_field, v_field, grid) # div(u,v) with vector halo exchange
curl_z(u_field, v_field, grid)    # vorticity: dv/dx - du/dy
laplacian(field, grid)            # 2nd-order  $\nabla^2$ 
```

```

advect_upwind(q, u, v, grid)      # 1st-order upwind advection
advect_centered(q, u, v, grid)    # 2nd-order centered advection
hyperdiffusion(field, grid, coeff) # 4th-order  $\nabla^4$  damping (two Laplacians)
global_integral(field, grid)      # Area-weighted integral (MPI-aware via allreduce)
global_mean(field, grid)         # Area-weighted mean

```

3D versions (`operators_3d.py`): vmapped over vertical levels on `(6, n, n, nlev)` arrays.

3.3.2 Centered Operators — Lat-Lon (`core/operators_latlon.py` , `operators_latlon_3d.py`)

Same operator set adapted for lat-lon geometry on `(n_lat, n_lon)` arrays: - Periodic in longitude, zero-gradient at poles - Metric terms include `cos(lat)` factors for spherical geometry - 3D versions via `vmap`

3.3.3 C-D Grid Operators — Cubed-Sphere (`core/operators_cdgrid.py`)

This is the canonical cubed-sphere operator module, shared by all atmosphere (SW, PE, CE) and ocean dynamical cores on the cubed-sphere. Uses FV3-style C-D grid staggering (Lin 2004, Putman & Lin 2007):

- **D-grid** winds (cell corners, `(6, n+1, n+1, ...)`) are prognostic for momentum
- **C-grid** velocities (cell edges) are diagnosed for mass/scalar transport
- **Vorticity** from circulation integral at D-grid corners (exact, no Hollingsworth-Kallberg instability)
- **Bernoulli gradient** via Arakawa-Lamb 4-point formula at D-grid corners

```

dgrid_to_cgrid(u_d, v_d, cdgrid)      # D-grid → C-grid interpolation
cgrid_to_dgrid(u_c, v_c, cdgrid)      # C-grid → D-grid
dgrid_vorticity(u_d, v_d, cdgrid)      # Circulation-based vorticity
cgrid_divergence(u_c, v_c, cdgrid)     # Divergence at cell centres
cgrid_mass_flux_divergence(h, u_c, v_c, cdgrid) # Upwind mass transport
_arakawa_lamb_gradient(B, cdgrid)      # Bernoulli/PGF gradient
cdgrid_momentum_tendencies(...)        # Full momentum tendency

```

3D versions (vmapped over vertical levels): `dgrid_vorticity_3d` , `cgrid_divergence_3d` , `cgrid_mass_flux_divergence_3d` , `_arakawa_lamb_gradient_3d` , `cdgrid_momentum_tendencies_3d` .

Relationship to FV3 (Lin 2004, Harris et al. 2022):

Feature	FV3 (GFDL)	legoESM C-D Grid
Grid staggering	C-D grid	C-D grid (same)
Transport	Flux-form semi-Lagrangian (FFSL)	Eulerian upwind (CFL < 1)
Momentum	Vorticity transported as scalar on D-grid	Circulation-based vorticity at D-grid corners
Vertical	Vertically Lagrangian with remapping	Eulerian σ or z^*
Splitting	Directional split (alternating x-y sweeps)	Unsplit
Damping	Divergence damping (2nd + 4th order)	Laplacian viscosity + hyperdiffusion

The C-D grid staggering eliminates the Hollingsworth-Kallberg instability that afflicts A-grid solvers, while maintaining JAX differentiability. State is stored on the A-grid for physics compatibility; conversion to/from D-grid occurs only in the momentum equation.

3.3.3b Finite-Volume Operators — Cubed-Sphere A-Grid (`core/operators_fv.py` , `operators_3d.py`)

PPM (Piecewise Parabolic Method) transport on A-grid cubed-sphere (supplementary to C-D grid):

```
fv_flux_divergence(q, u, v, grid) # Conservative flux-form transport
fv_scalar_advection(q, u, v, grid) # Advective (non-conservative) transport
```

Key design decisions: - **Unsplit**: both x and y fluxes computed on the SAME unmodified field - **PPM reconstruction** with Colella-Woodward limiter for monotonicity - **Reusable across atmosphere and ocean**: same operators used for atmospheric scalar transport and ocean tracer (T/S) advection

3D versions (`operators_3d.py`): `fv_flux_divergence_3d` , `fv_scalar_advection_3d` via `jax.vmap` .

3.3.4 Finite-Volume Operators — Lat-Lon (`core/operators_fv_latlon.py` , `operators_fv_latlon_3d.py`)

PPM operators adapted for lat-lon grid (same A-grid PPM approach as cubed-sphere): - Periodic boundary conditions in longitude - Zero-gradient extrapolation at latitude poles - Same unsplit design as cubed-sphere variant - Reuses `_ppm_edge_values` and `_ppm_limit` from `operators_fv.py`

Critical A-grid lesson: On A-grid lat-lon, PPM mass flux is energy-inconsistent with centered momentum operators. For shallow water, use centered divergence for mass (consistent with momentum). For PE/CE, use PPM for scalar transport (T, p_s, theta, rho, tracers) where feedback to momentum is indirect.

3.3.5 FC-Gram Operators (`core/operators_fc.py` , `operators_fc_3d.py` , `fc_gram.py`)

Fourier Continuation (FC-Gram) spectral operators for high-order derivatives on the cubed-sphere A-grid. FC-Gram extends periodic Fourier methods to non-periodic domains by appending a smooth continuation, enabling spectral accuracy at face interiors with controlled Gibbs artifacts at boundaries.

```
fc_gradient_x(field, grid, fc_basis) # FC spectral d/dx
fc_gradient_y(field, grid, fc_basis) # FC spectral d/dy
fc_divergence(u, v, grid, fc_basis)  # FC spectral divergence
fc_curl_z(u, v, grid, fc_basis)      # FC spectral vorticity
fc_laplacian(field, grid, fc_basis)  # FC spectral  $\nabla^2$ 
```

The `fc_gram.py` module constructs the FC continuation basis (Gram polynomial extension) and precomputes differentiation matrices. 3D versions vmapped over vertical levels.

3.3.6 Halo Exchange

Cubed-Sphere (`grids/halo.py` , `parallel/halo_exchange.py`):- `pad_halo(data)` $\rightarrow$ (6, n+2, n+2) — scalar halo padding - `pad_halo_vector(u, v, ...)` $\rightarrow$ vector halo with grid-angle rotation at face boundaries - MPI backend: edges packed per neighbor rank to reduce MPI messages

Lat-Lon (`grids/halo_latlon.py`):- Periodic in longitude, zero-gradient at poles - Simpler connectivity than cubed-sphere (no face rotations)

3.4 Time Integration

3.4.1 Integrator Interface

```
class TimeIntegrator(Protocol):
    def step(
        self,
        state: State,
        tendencies_fn: Callable[[State], State],
        dt: float,
    ) -> State: ...
```

3.4.2 Implemented Integrators

Integrator	File	Use Case
SSP-RK3	ssp_rk3.py	Shallow water, hydrostatic PE, ocean spectral
SSP-RK3(4)	ssp_rk34.py	Higher-order SSP with embedded error estimate
SSP-RK5(4)	ssp_rk54.py	5th-order SSP with 4th-order embedding
Split-explicit RK3	split_explicit.py	Non-hydrostatic (acoustic substeps), ocean (barotropic substeps)
Semi-implicit	semi_implicit.py	Hoskins-Simmons (1975) for spectral PE (Gamma corrected v3.1, RAW filter v3.7)
Tridiagonal solver	tridiagonal.py	Semi-implicit acoustic substeps (vertical), vertical diffusion

Robert-Asselin-Williams (RAW) filter (`semi_implicit.py`): The leapfrog time filter uses the Williams (2009) modification that splits the correction between the current and next time levels:

```
d_n = (gamma/2) * (X^{n-1} - 2*X^n + X^{n+1})
X^n_filtered = X^n + (1 - alpha) * d_n
X^{n+1}_filtered = X^{n+1} + alpha * d_n
```

With `alpha=0.5` (default), this restores second-order accuracy while preserving computational-mode damping. The original RA filter (`alpha=0`) introduces a first-order phase error that causes slow energy drift over climate-length integrations.

Adaptive hyperdiffusion (`core/cfl.py`): `adaptive_hyperdiff_coeff(dx_min, dt, order, safety)` computes the maximum stable hyperdiffusion coefficient from the diffusion CFL condition $\nu * dt / dx^n < C(n)$, eliminating manual tuning.

3.4.3 SSP-RK3

```
k1 = state + dt * F(state)
k2 = 3/4 * state + 1/4 * (k1 + dt * F(k1))
k3 = 1/3 * state + 2/3 * (k2 + dt * F(k2))
```

3.4.4 Split-Explicit Strategy

For non-hydrostatic atmosphere and ocean: - **Slow tendencies**: Coriolis, advection, horizontal PGF, diffusion (once per RK3 stage) - **Fast substeps**: Acoustic (atmosphere) or barotropic (ocean) modes via `fori_loop` or `scan` - `dt_fast = dt_slow / N_substeps` (atmosphere: N=6 default, ocean: N=30 default) - **Acoustic off-centering** (v3.7): `acoustic_off_centering` parameter (beta) applies $(1+\text{beta}) \cdot \text{rho_new} - \text{beta} \cdot \text{rho_old}$ to the density update, damping vertically-propagating acoustic/gravity wave noise without tightening the horizontal CFL (Skamarock & Klemp 2008). Default beta=0; recommended 0.1 for long runs.

3.5 Conservation

Conservation is a **hard constraint** enforced via projection after each timestep.

3.5.1 Conserved Quantities

Quantity	Method	Scope
Dry air mass	Surface pressure fixer (global integral preserved)	Global
Total water mass	Column-wise: <code>sum(q_v + q_c + q_i + q_r + q_s) * dp/g</code> preserved	Column + Global
Total energy	Global fixer: kinetic + internal + potential + latent	Global
Momentum	Angular momentum conservation via Coriolis discretization	Global
Tracer mass	Positive-definite flux-form advection (Zalesak limiter)	Global per tracer

3.5.2 Conservation Fixer

Applied after each full timestep:

```
def apply_conservation_fixer(state_new: State, state_old: State, grid: Grid) -> State:
    """Project state_new onto the conservation manifold."""

    # 1. Compute global integrals
    mass_old = global_integral(state_old.p_surface, grid)
    mass_new = global_integral(state_new.p_surface, grid)

    # 2. Apply uniform correction (preserves gradients for differentiability)
    correction = (mass_old - mass_new) / grid.total_area
    p_surface_fixed = state_new.p_surface + correction

    # 3. Similarly for energy and moisture
    return state_new.replace(p_surface=p_surface_fixed, ...)
```

The fixer uses **uniform additive corrections** (not multiplicative) to preserve gradients for automatic differentiation.

Initial-mass anchoring (v3.7): For long AMIP/CMIP integrations, the conservation fixer can anchor to the initial mass rather than the previous step's mass, preventing cumulative $O(\epsilon)$ drift over millions of timesteps. Call `model.set_initial_mass(state)` before the integration loop. Without anchoring, the fixer corrects to the previous step and $O(\epsilon)$ errors accumulate.

Grid-portable energy diagnostic (v3.7): `compute_hydrostatic_energy` now uses `p_s[... , None] * dsigma / g` broadcasting, which works for both cubed-sphere `(6, n, n, nlev)` and lat-lon `(n_lat, n_lon, nlev)` state shapes.

3.6 Smooth Approximations (`core/smooth.py`)

All discontinuous operations are replaced with smooth, differentiable alternatives:

```
def sigmoid_switch(x, sharpness=100.0):
    """Smooth approximation to Heaviside step function."""
    return jax.nn.sigmoid(sharpness * x)

def smooth_max(a, b, sharpness=100.0):
    """Differentiable approximation to max(a, b)."""
    return jax.nn.logsumexp(jnp.stack([a * sharpness, b * sharpness]), axis=0) / sharpness

def smooth_min(a, b, sharpness=100.0):
    """Differentiable approximation to min(a, b)."""
    ...

def smooth_clamp(x, lo, hi, sharpness=100.0):
    """Differentiable clamp."""
    return smooth_max(smooth_min(x, hi, sharpness), lo, sharpness)
```

4. Component Specifications

4.1 Atmosphere

4.1.1 Dynamical Cores Overview

The atmosphere has **24+ implemented dynamical cores** spanning five discretization families (C-D grid FV3, finite-volume PPM, pseudospectral, FC-Gram spectral, C-grid), three grid types (cubed-sphere, lat-lon, Gaussian), three equation sets (shallow water, hydrostatic PE, non-hydrostatic compressible Euler), and learned (SFNO) variants.

Architecture note (v3.7): All cubed-sphere solvers default to the C-D grid FV3 variant. Legacy names (`CompressibleEulerModel` , `ShallowWaterModel` , `PrimitiveEquationModel`) resolve to their `CDGrid` counterparts. The same `operators_cdgrid` module is shared across atmosphere (SW, PE, CE) and ocean.

C-D Grid FV3 + Spectral + Lat-Lon + Learned (15 models):

Model	Grid	Equations	Discretization	Time Integration
<code>CDGridShallowWaterModel</code>	Cubed-sphere	Shallow water	C-D grid FV3	SSP-RK3
<code>FVShallowWaterModel</code>	Cubed-sphere	Shallow water	C-D grid (wrapper)	SSP-RK3
<code>FVShallowWaterLatLonModel</code>	Lat-lon	Shallow water	FV (PPM)	SSP-RK3

Model	Grid	Equations	Discretization	Time Integration
SpectralShallowWaterModel	Gaussian	Shallow water (vor-div)	Spectral	SSP-RK3
SFNOShallowWaterModel	Gaussian	Shallow water	Learned (SFNO)	—
CDGridPrimitiveEquationModel	Cubed-sphere	Hydrostatic PE (σ)	C-D grid FV3	SSP-RK3
FVLatLonPrimitiveEquationModel	Lat-lon	Hydrostatic PE (σ)	FV (PPM)	SSP-RK3
SpectralPrimitiveEquationModel	Gaussian	Hydrostatic PE (vor-div- σ)	Spectral	SSP-RK3
SFNOPrimitiveEqModel	Gaussian	Hydrostatic PE	Learned (SFNO)	—
CDGridCompressibleEulerModel	Cubed-sphere	Non-hydrostatic (z^*)	C-D grid FV3	Split-explicit RK3
CompressibleEulerModel (A-grid)	Cubed-sphere	Non-hydrostatic (z^*)	Centered (legacy)	Split-explicit RK3
FVCompressibleEulerLatLonModel	Lat-lon	Non-hydrostatic (z^*)	FV (PPM)	Split-explicit RK3
SpectralCompressibleEulerModel	Gaussian	Non-hydrostatic (vor-div- z^*)	Spectral	Split-explicit RK3
MPASPrimitiveEquationModel	Voronoi/MPAS	Hydrostatic PE	TRiSK	SSP-RK3
MPASCompressibleEulerModel	Voronoi/MPAS	Non-hydrostatic (z^*)	TRiSK	Split-explicit RK3

FC-Gram spectral (6 models):

Model	Grid	Equations	Discretization	Time Integration
FCShallowWaterModel	Cubed-sphere	Shallow water	FC-Gram	SSP-RK3
FCCGShallowWaterModel	Cubed-sphere	Shallow water	FC-Gram + div damping	SSP-RK3
FCPrimitiveEquationModel	Cubed-sphere	Hydrostatic PE (σ)	FC-Gram	SSP-RK3
FCCGPrimitiveEquationModel	Cubed-sphere	Hydrostatic PE (σ)	FC-Gram + div damping	SSP-RK3
FCCompressibleEulerModel	Cubed-sphere	Non-hydrostatic (z^*)	FC-Gram	Split-explicit RK3
FCCGCompressibleEulerModel	Cubed-sphere	Non-hydrostatic (z^*)	FC-Gram + div damping	Split-explicit RK3

C-grid / divergence-damped (4 models):

Model	Grid	Equations	Discretization	Time Integration
CGShallowWaterLatLonModel	Lat-lon	Shallow water	True C-grid	SSP-RK3
CGShallowWaterCubedModel	Cubed-sphere	Shallow water	A-grid + div damping	SSP-RK3
CGPrimitiveEquationModel	Cubed-sphere	Hydrostatic PE (σ)	A-grid + div damping	SSP-RK3
CGCompressibleEulerModel	Cubed-sphere	Non-hydrostatic (z^*)	A-grid + div damping	Split-explicit RK3

All models follow the same API: `state = model.step(state, dt)`, with `integrate()` and `integrate_scan()` (differentiable via `lax.scan`) methods. The factory function `create_model(equations, discretization)` selects from 24 model variants via a two-axis key system.

4.1.2 Shallow Water Equations

Vector-invariant form (shared across all grids):

$$\begin{aligned} dh/dt &= -\text{div}(h \cdot v) \\ du/dt &= (\zeta + f) \cdot v - \partial B / \partial x + D_u \\ dv/dt &= -(\zeta + f) \cdot u - \partial B / \partial y + D_v \end{aligned}$$

where $B = K + g(h + h_s)$, $K = \frac{1}{2}(u^2 + v^2)$, $\zeta = \partial v / \partial x - \partial u / \partial y$.

State: `ShallowWaterState(h, u, v, h_s)`. **Config:** `g=9.81`, `hyperdiff_coeff=0.0`, conservation fixer flags.

FV variants use PPM for mass transport and PPM-compatible gradients for Bernoulli gradient. On lat-lon A-grid, mass uses centered divergence for energy consistency.

4.1.3 Hydrostatic Primitive Equations

σ -coordinate PE in vector-invariant form:

$$\begin{aligned} dp_s/dt &= -p_s \cdot \Sigma[\text{div}(v_k) \cdot \Delta \sigma_k] / (1 - \sigma_{\text{top}}) \\ du/dt &= (\zeta + f) \cdot v - \partial B / \partial x - R_d \cdot T \cdot \partial(\ln p_s) / \partial x \\ dT/dt &= -v \cdot \nabla T - \dot{\sigma} \cdot \partial T / \partial \sigma + \kappa \cdot T \cdot \omega / p \end{aligned}$$

Geopotential via Simmons-Burridge hydrostatic integration with α_k correction. Sigma-dot diagnosed from continuity with proper BCs ($\dot{\sigma}=0$ at top/bottom).

State: `HydrostaticState(u, v, T, p_s, phis)` — 3D fields `(6,n,n,nlev)` or `(n_lat,n_lon,nlev)`, surface fields `(6,n,n)` or `(n_lat,n_lon)`.

FV variants use PPM for T and `p_s` transport, centered operators for momentum.

4.1.4 Non-Hydrostatic Compressible Euler

Height z^* coordinates with reference-state subtraction to avoid cancellation:

```

dp'/dt = -(1/J)[div_h(J·p·v_h) + ∂(p·w)/∂z*]
dθ'/dt = -v·∇θ - (w/J)·∂θ/∂z*
du/dt = (ζ+f)·v - ∂K/∂x - c_p·θ·∂π'/∂x
dw/dt = -c_p·θ·(1/J)·∂π'/∂z* - g·θ'/θ_0

```

Exner perturbation: $\pi' = \pi_0 \cdot [((1 + \rho'/\rho_0)(1 + \theta'/\theta_0))^{(R_d/c_v)} - 1]$ (ratio form).

Split-explicit time stepping: RK3 outer loop (slow tendencies: Coriolis, horizontal PGF, advection, hyperdiffusion, sponge) with N acoustic substeps (vertical PGF, buoyancy, continuity).

State: NonHydrostaticState(u, v, w, theta_prime, rho_prime,phis, tracers). **Config:**
n_acoustic_substeps=6, sponge_width=10000m, sponge_coeff=0.05, acoustic_off_centering=0.0
(set to 0.1 for long AMIP/CMIP runs).

4.1.5 Spectral Variants

Pseudospectral vorticity-divergence formulation on the Gaussian grid (Bourke 1972):

```

∂ζ/∂t = -div((ζ+f)·v) + curl(F_friction)
∂D/∂t = curl((ζ+f)·v) - ∇²(K+Φ) - R_d·T_ref·∇²(lnp_s) - ∇·(R_d·T'·∇lnp_s) +
div(F_friction)
∂T/∂t = -v·∇T + κ·T·(σ/σ - v·∇lnp_s - D) - σ·∂T/∂σ
∂lnp_s/∂t = -Σ(D_k·Δσ_k) / (1-σ_top)

```

Workflow: SH synthesis → grid-space nonlinear products → SH analysis → spectral tendencies. Spectral hyperdiffusion: $-v \cdot [n(n+1)/a^2]^{\text{order per coefficient}}$. Metal backend: automatic CPU fallback for complex128 SH transforms.

Pressure gradient force: Uses the mathematically correct form $-\nabla^2(K+\Phi) - \nabla \cdot (R_d \cdot T \cdot \nabla \ln p_s)$ split as $-\nabla^2(K+\Phi) - R_d \cdot T_{\text{ref}} \cdot \nabla^2(\ln p_s) - \nabla \cdot (R_d \cdot T' \cdot \nabla \ln p_s)$ where $T' = T - T_{\text{ref}}$. The T_{ref} subtraction (Simmons & Burridge 1981) eliminates spectral transform cancellation errors. The correction term $\nabla \cdot (R_d \cdot T' \cdot \nabla \ln p_s)$ is computed in grid space (product of grid fields) and transformed to spectral via the divergence operator.

Bug fix (v3.1): The original code used the Bourke (1972) E-variable form $E = K + \Phi + R_d \cdot T \cdot \ln p_s$, which adds a spurious same-level coupling $-R_d \cdot \ln p_{s0} \cdot \nabla^2(T')$ to the divergence equation. Since $-\nabla^2(R_d \cdot T \cdot \ln p_s) \neq -\nabla \cdot (R_d \cdot T \cdot \nabla \ln p_s)$, the E-variable form is only valid when $\nabla \ln p_s \approx 0$. With the reference surface pressure $\ln p_{s0} \approx 11.51$, this creates exponential instability via the coupling chain $D \rightarrow T$ (adiabatic) $\rightarrow D$ (spurious PGF), with stability criterion $\kappa \cdot (\alpha_{SB} + \ln p_{s0}) = 3.49 > 1$. The correct PGF form eliminates this instability entirely — eigenvalue analysis confirms purely imaginary eigenvalues (gravity waves, no growth). The semi-implicit Gamma matrix (semi_implicit.py) was also corrected to remove the corresponding $\ln p_{s0} \cdot I$ diagonal.

Semi-implicit scheme: Hoskins & Simmons (1975) treats fast gravity-wave terms implicitly in the divergence equation. Precomputed LU factorizations per wavenumber n , applied after each RK stage.

4.1.6 Learned (SFNO) Variants

Spherical Fourier Neural Operator models for SW and PE: - Channel packing: state variables → ML channels and back - Trained via differentiable rollout loss - Drop-in replacement for physics-based dynamics via standard model API

4.1.7 FC-Gram Variants

Fourier Continuation (FC-Gram) spectral operators on the cubed-sphere grid, providing spectral-like accuracy for horizontal derivatives without requiring a Gaussian grid.

Core operators (`core/operators_fc.py` , `operators_fc_3d.py` , `fc_gram.py`): - FC-Gram basis construction and extension matrices - High-order gradient, divergence, curl, and Laplacian via FC spectral differentiation - 3D versions vmapped over vertical levels

Models (3 equation sets × 2 damping variants = 6 models):

Model	File	Description
<code>FCShallowWaterModel</code>	<code>shallow_water_fc.py</code>	FC-Gram SWE on cubed-sphere
<code>FCCGShallowWaterModel</code>	<code>shallow_water_fc_cgrid.py</code>	FC-Gram SWE + divergence damping
<code>FCPrimitiveEquationModel</code>	<code>primitive_eq_fc.py</code>	FC-Gram hydrostatic PE
<code>FCCGPrimitiveEquationModel</code>	<code>primitive_eq_fc_cgrid.py</code>	FC-Gram PE + divergence damping
<code>FCCompressibleEulerModel</code>	<code>compressible_euler_fc.py</code>	FC-Gram non-hydrostatic CE
<code>FCCGCompressibleEulerModel</code>	<code>compressible_euler_fc_cgrid.py</code>	FC-Gram CE + divergence damping

The FC-Gram + divergence damping variants combine FC spectral operators with C-grid-style 2nd + 4th order divergence damping to selectively dissipate divergent modes (barotropic gravity waves) while preserving rotational flow. Divergence damping is particularly valuable at cubed-sphere face boundaries where acoustic modes can alias.

Ocean variant: `ocean_pe_fc_cgrid.py` — FC-Gram ocean PE with divergence damping.

Factory selection: `discretization="fc_gram"` (pure FC) or `discretization="fc_gram_cgrid"` (FC + divergence damping).

4.1.8 C-Grid Variants

Two C-grid strategies are implemented for different grid types:

True C-grid on lat-lon (`shallow_water_cgrid_latlon.py`): - Full C-grid staggering: scalars (h) at cell centers, u at longitude interfaces, v at latitude interfaces - State:

`CGShallowWaterState(h, uc, vc, h_s)` — uc shape `(n_lat, n_lon)` , vc shape `(n_lat+1, n_lon)` - Exact pressure gradient force (adjacent-cell differencing, no $2\Delta x$ blind spot) - PPM mass transport at staggered interfaces - 4-point Coriolis averaging (interpolation from centers to edges) - 2nd + 4th order

divergence damping - Conversion utilities: `a_to_cgrid()`, `cgrid_to_a()` for interop with A-grid
 diagnostics - Helper `_ppm_face_values_1d(q_pad, n_out)` : reusable 1D PPM returning (a_L, a_R)

A-grid + divergence damping on cubed-sphere (`shallow_water_cgrid.py`, `primitive_eq_cgrid.py`, `compressible_euler_cgrid.py`) : - Uses A-grid storage (avoids staggered halo exchange complexity at face boundaries) - Adds FV3-style divergence damping (2nd + 4th order) to selectively dissipate divergent modes - PPM mass/scalar transport for conservation - Momentum remains vector-invariant (centered operators) - Compact inner ∇^2 for hyperdiffusion (resolves $2\Delta x$ checkerboard)

Model	File	Grid	Staggering
CGShallowWaterLatLonModel	<code>shallow_water_cgrid_latlon.py</code>	Lat-lon	True C-grid
CGShallowWaterCubedModel	<code>shallow_water_cgrid.py</code>	Cubed-sphere	A-grid + div damp
CGPrimitiveEquationModel	<code>primitive_eq_cgrid.py</code>	Cubed-sphere	A-grid + div damp
CGCompressibleEulerModel	<code>compressible_euler_cgrid.py</code>	Cubed-sphere	A-grid + div damp

Williamson TC2 results (48×96 lat-lon, 5 days): A-grid $L2=3.8e-4$, C-grid $L2=3.2e-3$. A-grid wins on balance preservation (centered div = exact discrete geostrophic balance). C-grid slightly better on TC5 KE conservation (17% vs 18%).

Factory selection: `discretization="cgrid"` (cubed-sphere div-damped) or `grid="latlon"` with `discretization="cgrid"` (true C-grid lat-lon).

4.1.9 Tracer Transport (`atmosphere/dynamics/tracer_transport.py`)

Passive tracer advection framework supporting arbitrary number of tracers.

4.1.10 Edge Blending (`atmosphere/dynamics/edge_blending.py`)

Utilities for smooth blending at cubed-sphere face boundaries.

4.1.11 Physics Parameterizations

All physics parameterizations follow a **standard factory pattern**:

```

config.py      - NamedTuple configs for each scheme
output.py     - NamedTuple output container (tendencies + diagnostics)
<scheme>.py   - Column physics: (ncol, nlev) → Output
integration.py - make_*_physics() factory: (config, model_type, dt) → physics_fn
  
```

Integration signatures by model type: - **Hydrostatic:** (state, grid, sigma_coord) → HydrostaticTendencies - **NonHydrostatic:** (state, grid, height_coord, terrain_metric) → NonHydrostaticTendencies - **Spectral PE:** (state, grid, sigma_coord) → SpectralHydrostaticState

Radiation (`atmosphere/physics/radiation/`)

Scheme	File	Description
Gray	<code>gray.py</code>	Frierson (2006) two-stream gray radiation

Scheme	File	Description
RRTMGP	<code>rrtmcp_radiation.py</code>	Full correlated-k radiative transfer (bundled jax-rrtmcp)

RRTMGP bundle (`rrtmcp/`): Complete JAX port of RTE-RRTMGP including gas optics, cloud optics, shortwave/longwave solvers, and configuration management.

Supporting: `solar.py` — solar geometry (zenith angle, insolation).

Convection (`atmosphere/physics/convection/`)

Scheme	File	Description
SBM	<code>sbm.py</code>	Simplified Betts-Miller (Frierson 2007) — cloud-layer masked
DCA	<code>dca.py</code>	Deep Convective Adjustment
Kuo	<code>kuo.py</code>	Kuo (1965/1974) moisture convergence
Mass-flux	<code>mass_flux.py</code>	Prognostic mass-flux (Arakawa-Wu)
EDMF	<code>edmf.py</code>	Eddy-diffusivity mass-flux

Bug fix (v3.1): SBM convection originally adjusted the entire column toward a moist-adiabatic reference profile, including the convectively stable stratosphere. Combined with a Newton enthalpy correction that added a uniform temperature offset to all levels, this created artificial CAPE and triggered convective cooling of ~40 K/hr at the model top ($\sigma=0.035$), causing blowup within hours. The fix adds a `cloud_mask = (T_moist  $\geq$  T_env)` that restricts the adjustment to levels where the moist adiabat indicates conditional instability (the convective layer). CAPE is now computed from the raw moist adiabat before the Newton correction, and the enthalpy constraint is applied only over the masked cloud layer.

Microphysics (`atmosphere/physics/microphysics/`)

Scheme	File	Description
Kessler	<code>kessler.py</code>	Warm-rain one-moment (Kessler 1969)
Sundqvist	<code>sundqvist.py</code>	Large-scale condensation (Sundqvist 1989)
Seifert-Beheng	<code>seifert_beheng.py</code>	Two-moment warm rain (Seifert & Beheng 2001)
Morrison	<code>morrison.py</code>	Double-moment ice+liquid (Morrison et al. 2005)
Thompson	<code>thompson.py</code>	Hybrid moment with graupel (Thompson et al. 2008)
ML emulator	<code>ml_emulator.py</code>	Neural network surrogate (Equinox MLP)

Turbulence / Boundary Layer (`atmosphere/physics/turbulence/`)

Scheme	File	Description
Smagorinsky	<code>smagorinsky.py</code>	Constant eddy diffusivity
Louis	<code>louis.py</code>	Stability-dependent diffusion (Louis 1979)

Scheme	File	Description
TKE/MY2.5	<code>tke.py</code>	Prognostic TKE, Mellor-Yamada 2.5 closure
CLUBB-lite	<code>clubb_lite.py</code>	CLUBB-lite skeleton (delegates to TKE)
Holtslag-Boville	<code>holtslag_boville.py</code>	Nonlocal K-profile with counter-gradient
YSU	<code>ysu.py</code>	YSU nonlocal K-profile with entrainment
EDMF	<code>edmf.py</code>	EDMF unified boundary layer framework
ML emulator	<code>ml_emulator.py</code>	Neural network surrogate

Supporting: `surface_layer.py` (Monin-Obukhov surface layer with MOST/COARE3/LY04 dispatch), `vertical_diffusion.py` (vertical diffusion driver), `pbl_height.py` (bulk Richardson PBL height diagnosis — all backends return `h_pbl` in `TurbulenceOutput`).

Gravity Wave Drag (`atmosphere/physics/gravity_wave_drag/`)

Scheme	File	Description
Rayleigh	<code>rayleigh.py</code>	Rayleigh friction drag
Lindzen	<code>lindzen.py</code>	Lindzen (1981) orographic GWD
McFarlane	<code>mcfarlane.py</code>	McFarlane (1987) orographic GWD
Hines	<code>hines.py</code>	Hines (1997) Doppler-spread non-orographic
Prognostic spectral	<code>prognostic_spectral.py</code>	Multi-azimuthal prognostic spectral GWD
ML emulator	<code>ml_emulator.py</code>	Neural network surrogate

Combined Physics (`atmosphere/physics/combined.py`)

`PhysicsConfig` + `make_physics()` — assembles a complete physics suite from individual module configs. Orchestrates radiation → convection → microphysics → turbulence → GWD sequentially.

Thermodynamics (`atmosphere/physics/thermodynamics.py`)

Shared thermodynamic utilities: saturation vapor pressure, virtual temperature, moist static energy, CAPE/CIN, etc.

Idealized Forcings

Forcing	File	Description
Held-Suarez	<code>held_suarez.py</code>	Newtonian relaxation + Rayleigh friction (spectral + cubed-sphere)
Held-Suarez (lat-lon)	<code>held_suarez_latlon.py</code>	Held-Suarez adapted for lat-lon grid
Baroclinic wave	<code>baroclinic_wave.py</code>	Jablonowski-Williamson initial conditions

4.2 Ocean

4.2.1 Overview

Boussinesq hydrostatic ocean primitive equation solver with: - Wright (1997) equation of state - z-star vertical coordinate with dynamic Jacobian - Split-explicit barotropic/baroclinic time stepping - Cubed-sphere FV, spectral (Gaussian grid), and learned (SFNO) discretizations - Land masking via boolean mask + zero-fill - Global ocean support (C48 ~2°, 50 vertical levels) - Comprehensive physics suite (mixing, convection, surface forcing, bottom drag)

Architecture: `OceanModel` (FV on cubed-sphere), `SpectralOceanModel` (pseudospectral on Gaussian grid), `SFNOOceanModel` (learned).

4.2.2 Governing Equations

Boussinesq hydrostatic primitive equations in vector-invariant form:

$$\begin{aligned} du/dt &= (\zeta+f) \cdot v - \partial B / \partial x - (1/\rho_0) \cdot \partial p' / \partial x + A_h \cdot \nabla^2 u + \partial / \partial z (A_v \cdot \partial u / \partial z) - w \cdot \partial u / \partial z \\ dv/dt &= -(\zeta+f) \cdot u - \partial B / \partial y - (1/\rho_0) \cdot \partial p' / \partial y + A_h \cdot \nabla^2 v + \partial / \partial z (A_v \cdot \partial v / \partial z) - w \cdot \partial v / \partial z \\ dT/dt &= -u \cdot \nabla T - w \cdot \partial T / \partial z + K_h \cdot \nabla^2 T + \partial / \partial z (K_v \cdot \partial T / \partial z) \\ dS/dt &= -u \cdot \nabla S - w \cdot \partial S / \partial z + K_h \cdot \nabla^2 S + \partial / \partial z (K_v \cdot \partial S / \partial z) \\ d\eta/dt &= -\Sigma_k \operatorname{div}(h_k \cdot v_k) \end{aligned}$$

Diagnostic relations: - Density: $\rho = \text{wright_eos}(T, S, p_{\text{hydro}})$ - Hydrostatic pressure: $p(z) = \rho_0 g \eta + \int_0^z \rho' g \, dz'$ (top-down cumsum) - Vertical velocity: $w(z) = -\int_{-H}^z \operatorname{div}(v) \, dz'$ (bottom-up, $w=0$ at floor) - Layer thickness: $h_k = dz_{\text{ref}}[k] \cdot J$, where $J = (\eta + H_{\text{bathy}}) / H_{\text{max}}$ - Kinetic energy: $B = \frac{1}{2}(u^2 + v^2)$ - Vorticity: $\zeta = \partial v / \partial x - \partial u / \partial y$

4.2.3 Equation of State (`ocean/eos.py`)

Wright (1997) 9-term polynomial EOS (J. Atmos. Oceanic Tech., 14(3), 735–740):

$$\rho = (p + p_0) / (\lambda + \alpha_0 \cdot (p + p_0))$$

where $\alpha_0(T,S)$, $p_0(T,S)$, $\lambda(T,S)$ are polynomial functions of temperature and salinity. Reference values: $T=25^\circ\text{C}$, $S=35$ PSU, $p=0 \rightarrow \rho \approx 1023.3 \text{ kg/m}^3$.

Additional functions: - `density_perturbation(T, S, p)` $\rightarrow \rho' = \rho - \rho_0$ - `compute_hydrostatic_pressure(rho, eta, dz, jacobian)` — top-down cumulative integral - `compute_buoyancy_frequency(rho, dz, jacobian)` $\rightarrow N^2 = -(g/\rho_0) \cdot dp/dz$

Ocean constants: $\rho_0 = 1025.0 \text{ kg/m}^3$, $c_{\text{sw}} = 3994.0 \text{ J/(kg}\cdot\text{K)}$.

4.2.4 State Containers (`ocean/state.py`)

FV Ocean State (cubed-sphere):

Field	Shape	Units	Description
u	(6,n,n,nlev)	m/s	Zonal velocity

Field	Shape	Units	Description
v	(6,n,n,nlev)	m/s	Meridional velocity
T	(6,n,n,nlev)	°C	Potential temperature
S	(6,n,n,nlev)	PSU	Salinity
eta	(6,n,n)	m	Sea surface height
H_bathy	(6,n,n)	m	Bathymetry depth (static, positive)
land_mask	(6,n,n)	0/1	Ocean=1, land=0 (static)

OceanConfig defaults:

Parameter	Default	Units	Purpose
g	9.80616	m/s ²	Gravity
rho_0	1025.0	kg/m ³	Reference density
A_h	1.0e4	m ² /s	Horizontal viscosity
K_h	1.0e3	m ² /s	Horizontal tracer diffusivity
A_v	1.0e-3	m ² /s	Vertical viscosity
K_v	1.0e-4	m ² /s	Vertical tracer diffusivity
n_barotropic_substeps	30	—	Barotropic subcycles per step

4.2.5 Split-Explicit Time Stepping

Baroclinic (slow) step (`ocean/dynamics/ocean_pe.py`): 1. Compute layer thickness h_k and Jacobian J from η , H_{bathy} 2. Density from Wright EOS and hydrostatic pressure (top-down cumsum) 3. Diagnose w from continuity (bottom-up integral of $\text{div}(\mathbf{v} \cdot \mathbf{h})$) 4. Coriolis split: planetary f on baroclinic shear, relative ζ on full velocity 5. Pressure gradient, vertical advection (upwind) 6. Tracer advection: centered (default) or PPM (`use_fv_tracer_transport=True`) 7. Horizontal/vertical mixing, optional hyperdiffusion 8. Land masking of all tendencies 9. Free-surface tendency from depth-integrated flux divergence

Tracer transport options: - **Centered** (default): $-(\mathbf{u} \cdot \partial \mathbf{T} / \partial \mathbf{x} + \mathbf{v} \cdot \partial \mathbf{T} / \partial \mathbf{y})$ — simple, consistent with momentum operators - **PPM** (`use_fv_tracer_transport=True`): Reuses the same `fv_scalar_advection_3d` operators as the atmospheric FV dynamics. Reduces spurious numerical mixing, important for maintaining sharp thermocline gradients. Momentum remains centered (vector-invariant form).

Barotropic (fast) substeps (`ocean/dynamics/barotropic.py`):

Forward-backward substeps for 2D free-surface gravity waves with semi-implicit Coriolis. Loop via `jax.lax.fori_loop` (fast) or `jax.lax.scan` (differentiable).

4.2.6 Ocean Physics

All ocean physics modules follow the same factory pattern as atmospheric physics.

Vertical Mixing (`ocean/physics/vertical_mixing/`)

Scheme	File	Description
Constant	<code>constant.py</code>	Constant diffusivity/viscosity
Richardson	<code>richardson.py</code>	Richardson number-dependent mixing
KPP	<code>kpp.py</code>	K-Profile Parameterization (Large et al.)

Lateral Mixing (`ocean/physics/lateral_mixing/`)

Scheme	File	Description
Harmonic	<code>harmonic.py</code>	Laplacian diffusion
Biharmonic	<code>biharmonic.py</code>	Biharmonic hyperdiffusion
GM-Redi	<code>gm_redi.py</code>	Gent-McWilliams/Redi isopycnal mixing

Surface Forcing (`ocean/physics/surface_forcing/`)

Scheme	File	Description
Prescribed	<code>prescribed.py</code>	Prescribed heat/freshwater fluxes
Restoring	<code>restoring.py</code>	Relaxation to target T/S profiles
Bulk formulas	<code>bulk_formulas.py</code>	Bulk aerodynamic flux computation (constant/MOST/COARE3/LY04)

Bottom Drag (`ocean/physics/bottom_drag/`)

Scheme	File	Description
Linear	<code>linear.py</code>	Linear bottom drag
Quadratic	<code>quadratic.py</code>	Quadratic bottom drag

Ocean Convection (`ocean/physics/convection/`)

Scheme	File	Description
Enhanced diffusion	<code>enhanced_diffusion.py</code>	Enhanced diffusivity for unstable columns
Plume	<code>plume.py</code>	Plume convection model

Combined Ocean Physics (`ocean/physics/combined.py`)

Assembles a complete ocean physics suite from individual module configs.

4.2.7 Simple Ocean (`ocean/simple_ocean.py`)

Slab ocean model for atmosphere-only experiments with interactive SST.

4.2.8 Conservation (ocean/conservation.py)

Quantity	Method
Volume	Uniform additive η correction: $\iint \eta \cdot \text{area} = \text{const}$
Heat	Uniform additive T correction: $\iiint T \cdot h_k \cdot \text{area} = \text{const}$
Salt	Uniform additive S correction: $\iiint S \cdot h_k \cdot \text{area} = \text{const}$

MPI-aware global sums via `global_sum_mpi()` in distributed mode.

4.2.9 Validated Test Cases

Test	Resolution	Duration	Validation
Stommel gyre	C16–C32	300 days	Steady western boundary current
Baroclinic adjustment	C16–C32	300 days	Thermocline flattening
Equatorial Kelvin wave	C16–C32	300 days	Eastward propagation
Rest state	C8	50 steps	Tendencies < 1e-6, T/S bounded
Differentiability	C8	1 step	<code>jax.grad</code> through <code>model.step</code> produces finite gradients

4.2.10 Ocean Biogeochemistry (ocean/biogeochemistry/)

Two schemes controlled by `BiogeoConfig.scheme` :

Abiotic (`scheme="abiotic"`) : - Prognostic DIC and ALK (dissolved inorganic carbon, total alkalinity) - Carbonate equilibria solver (CO_2 solubility, $\text{HCO}_3^-/\text{CO}_3^{2-}$ speciation) - Air-sea CO_2 gas exchange: Wanninkhof (2014) parameterization with Schmidt number and gas transfer velocity - `air_sea_co2_flux(pCO2_ocean, pCO2_atm, T, S, U10)` [mol/m²/s]

NPZD ecosystem (`scheme="npzd"`) : - Four biological tracers: NO_3^- , phytoplankton, zooplankton, detritus (Fasham et al. 1990) - Phytoplankton growth: Michaelis-Menten nutrient limitation $\times$ light limitation $\times$ temperature-dependent maximum rate - Light model: Beer-Lambert PAR attenuation with water absorption + chlorophyll self-shading - Grazing: Holling-III functional response with assimilation efficiency - Detrital sinking with depth-dependent remineralization - Redfield stoichiometry coupling: $\text{R}_\text{C}:\text{N} = 6.625$, $\text{R}_\text{O}:\text{N} = 10.625$, CaCO_3 rain ratio = 0.07 - Full coupling to DIC/ALK via biological production and respiration

State: `OceanBiogeoState(DIC, ALK, N03, Phyto, Zoo, Det)` — all tracers shape `(6, n, n, nlev)`.

Key parameters: $\mu_\text{max} = 1.5/\text{day}$ (max growth), $k_\text{N} = 0.7 \text{ mmol/m}^3$ (half-saturation), $w_\text{sink} = 10 \text{ m/day}$ (detrital sinking), $\text{remin_rate} = 0.05/\text{day}$.

4.3 Land

4.3.1 Overview

Two land models: slab (`land/slab_land.py`) for simple experiments and multi-layer (`land/multilayer_land.py`) with Richards equation soil hydrology, soil thermal diffusion, and full surface energy balance. Both include snow budget and optional dynamic albedo via `surface_albedo.py` .

4.3.2 Slab Land Model

- Surface energy balance: $R_{\text{net}} = H + LE + G$
- Single-layer soil moisture bucket
- Snow budget: accumulation from precipitation, melt proportional to T above T_{melt}
- Optional snow-albedo feedback via `LandAlbedoConfig`
- `step_land(state, forcing, config, dt) → (LandState, TileResponse)`

4.3.3 Multi-Layer Land Model (`land/multilayer_land.py`)

- Multi-layer soil temperature (backward Euler thermal diffusion, Johansen 1975 conductivity)
- Multi-layer soil moisture (Richards equation, Celia et al. 1990 mixed-form Picard iteration)
- 6 retention curves: van Genuchten, Clapp-Hornberger, Brooks-Corey, Campbell, PDI, Lu
- Surface and subsurface runoff generation
- Snow budget with aging and dynamic albedo
- `step_multilayer_land(state, forcing, config, dt) → (MultiLayerLandState, TileResponse)`

4.3.4 Surface Albedo (`surface_albedo.py`)

Top-level module with all surface albedo parameterizations:

Function	Description
<code>land_vegetation_albedo(lat)</code>	Latitude-dependent (tropics 0.15, midlat 0.20, highlat 0.25)
<code>snow_albedo(age)</code>	Exponential aging decay (max 0.80 → min 0.50, $\tau = 5$ days)
<code>snow_cover_fraction(depth)</code>	Linear ramp to critical depth (50 kg/m ²)
<code>land_albedo(lat, snow, age)</code>	Vegetation + snow blending
<code>ice_albedo(T)</code>	Temperature-dependent sigmoid (cold 0.65 ↔ warm 0.45, $\Delta T = 5$ K)
<code>ocean_albedo(cos_zenith)</code>	Constant (0.06) or Briegleb 1992 zenith-dependent

Configs: `LandAlbedoConfig` , `IceAlbedoConfig` , `OceanAlbedoConfig` .

4.3.5 State (`land/state.py`)

```
class LandState(NamedTuple):
    T_soil: Field          # Soil temperature [K]
    W_bucket: Field        # Soil moisture [kg/m²]
    snow_depth: Field      # Snow water equivalent [kg/m²]
    snow_age: Field        # Time since last snowfall [s]
```

```
class MultiLayerLandState(NamedTuple):
    T_soil: jax.Array      # (ncol, n_layers)
    psi_soil: jax.Array    # Matric potential [m]
    theta_soil: jax.Array  # Volumetric water content
    runoff_surface: jax.Array
    runoff_subsurface: jax.Array
    snow_depth: jax.Array
    snow_age: jax.Array
```

4.3.6 Land Carbon Cycle (`land/carbon/`)

Two carbon cycle schemes integrated into both slab and multi-layer land:

- **DifferLand** (`carbon_cycle.py`): DALEC-990 six-pool model (labile, foliage, root, wood, litter, SOM). Light-use-efficiency GPP with DALEC phenology. Q10 decomposition with moisture limitation. Full carbon-water coupling.
- **Seasonal** (`carbon_cycle.py`): Sinusoidal NEE with latitude-dependent amplitude/phase (quick-look carbon diagnostics).

`step_land()` and `step_multilayer_land()` return 3-tuples: `(state, response, carbon_state)`.

4.3.7 Plant Physiology (`land/carbon/stomata.py`)

Stomatal conductance and photosynthesis module, activated when `StomataConfig.enabled = True`:

Farquhar (1980) C3 photosynthesis: - Rubisco-limited (W_c) and RuBP-regeneration-limited (W_j) rates with smooth minimum - Arrhenius and peaked-Arrhenius temperature responses (Bernacchi et al. 2001 parameters) - Soil moisture stress on $V_c\text{_{max}}$ (CLM approach: $V_c\text{_{max}} \times \beta_{\text{soil}}$)

Stomatal conductance models:

Model	Formula	Activation
Ball-Berry (1987)	$g_s = g_0 + g_1 \cdot A \cdot RH / C_s$	Carbon cycle active
Medlyn (2011)	$g_s = g_0 + 1.6 \cdot (1 + g_1 / \sqrt{VPD}) \cdot A / C_s$	Carbon cycle active
Jarvis (1976)	$g_s = g_{s_max} \cdot f(PAR) \cdot f(T) \cdot f(VPD) \cdot f(soil)$	Carbon cycle inactive

Coupled solver: Fixed-point A-gs-Ci iteration (5 steps, unrolled for JIT). Farquhar GPP passed as `gpp_override` to carbon cycle (avoids double-counting).

ET coupling: Beer-law canopy fraction blends bare-soil evaporation with canopy transpiration. `compute_stomatal_beta()` returns effective β for surface flux computation.

4.3.8 Future Phases

Remaining: - Dynamic LAI (currently prescribed) - Vegetation dynamics and competition - Carbon-nitrogen coupling - Groundwater and runoff routing

4.4 Cryosphere

4.4.1 Sea Ice — Three Dynamics Modes (`ice/sea_ice.py`)

The sea ice model supports three dynamics modes, controlled by `SeaIceConfig.dynamics`:

Slab (default, `dynamics="none"`): - Thermodynamic-only: growth/melt from surface energy balance - Free-drift diagnostic velocity (wind-driven) - `SeaIceState(h_ice, T_ice, concentration)` — backward compatible - Optional temperature-dependent albedo via `IceAlbedoConfig`

Free Drift (`dynamics="free_drift"`): - Diagnostic velocity from wind + ocean current - Optional tracer advection (ice area, volume)

EVP (`dynamics="evp"`): - Elastic-Viscous-Plastic rheology (Hunke & Dukowicz 1997) - Subcycled momentum solver: `N_evp` substeps per timestep (default 120) - Internal ice stress tensor from elliptic yield curve ($e = 2.0$) - Ice strength: $P = P \cdot h \cdot \exp(-C \cdot (1 - c))$ with $P = 27.5$ kPa, $C = 20$ - `DynamicSeaIceState` with `u_ice`, `v_ice`, `sigma_11/22/12` stress tensor fields - Coriolis, ocean tilt, air/ocean drag, internal stress divergence

4.4.2 Multi-Category Ice

When `SeaIceConfig.n_categories > 1` (CICE framework): - Per-category state: `h_cat`, `T_cat`, `c_cat` arrays (shape `[ncol, n_cat]`) - Linear remapping (Lipscomb 2001) redistributes ice across thickness categories after thermodynamic growth/melt - Category boundaries follow equal-area partitioning - Bulk properties (mean `h`, `T`, `c`) derived from category-weighted averages

4.4.3 Future Phases

- Ice sheets: Shallow-ice/shallow-shelf approximation
- Permafrost: Deep soil extension with phase change

4.5 Coupler

4.5.1 Current Implementation (`coupler/`)

The coupler is fully implemented with tile-based surface exchange:

Core coupling (`coupler/coupler.py`):

```
make_coupler(config) → coupler_fn(atm_state, surface_states, dt) → coupled_tendencies
```

Ocean albedo plumbing: `CouplerConfig.ocean_albedo` (float, default 0.06) controls the constant ocean albedo. When `ocean_albedo_config.method == "constant"`, the coupler overrides `OceanAlbedoConfig.alpha_ocean_const` with this value. When `method == "zenith"`, the Briegleb (1992) zenith-angle formula is used instead and `ocean_albedo` is ignored.

q_surface consistency: All slab surface tiles (land, sea ice, lake) recompute `q_surface` from the updated post-step surface temperature before constructing `TileResponse`. For slab land, the moisture availability factor β is also recomputed from updated bucket moisture. This ensures `q_surface` and `T_surface` in `TileResponse` are always thermodynamically consistent.

Coupling fields (`coupler/coupling_fields.py`): - `AtmToSurface` — downward radiation, precipitation, wind, temperature, humidity - `SurfaceToAtm` — sensible heat, latent heat, albedo, roughness, SST - `TileResponse` — per-tile surface response (13 fields including `T_surface`, `q_surface`, `albedo`)

Tile fractions (`coupler/tile_fractions.py`): - `TileFractions(ocean, land, lake, ice)` — fractional coverage per grid cell - `blend_tiles()` — area-weighted blending of tile responses

Surface exchange (`coupler/surface_exchange.py`): - Atmosphere-surface flux extraction (bulk aerodynamic formulas)

Flux accumulation (`coupler/accumulator.py`): - Time-averaged flux accumulation for coupling intervals > physics timestep

4.5.2 Lake Model (`coupler/lake/`)

Two-layer lake model (epilimnion + hypolimnion) for inland water bodies: - `LakeState(T_epi, T_hypo)` — epilimnion and hypolimnion temperatures - `LakeConfig` — layer depths, mixing coefficients, albedo, emissivity, bulk scheme selection - Epilimnion energy balance: $p \cdot c \cdot h_{\text{epi}} \cdot dT_{\text{epi}}/dt = SW_{\text{net}} + LW_{\text{net}} - SH - LH - F_{\text{mix}}$ - Wind-enhanced vertical mixing: $k_{\text{eff}} = k_{\text{mix}} \cdot (1 + \alpha \cdot |V|)$ - Supports constant or MOST bulk flux schemes - `step_lake(state, forcing, config, U_min, dt)` → (`LakeState`, `TileResponse`)

4.5.3 Bulk Flux Module (`coupler/bulk_flux.py`)

Unified Monin-Obukhov Similarity Theory (MOST) surface flux computation shared by all surface tiles:

```
compute_most_fluxes(
    u_rel, v_rel, T_atm, q_atm, T_sfc, q_sfc, rho,
    z_ref=10.0, z0_init=1e-4, scheme="coare3", n_iter=5, charnock=0.011,
) → (tau_x, tau_y, shflx, lhflx, u_star)
```

Three schemes:

Scheme	Roughness	Use Case
"most"	Fixed z0 (from caller)	Land, sea ice, lake
"coare3"	Charnock + smooth-flow: $z0 = \alpha_c \cdot u^2 / g + 0.11 \cdot \nu / u^*$	Ocean (Fairall et al. 2003)
"large_yeager"	Empirical C_DN(U_10N)	Ocean (Large & Yeager 2004 / CORE)

Key features: - Businger-Dyer stability functions $\psi_m(\zeta)$, $\psi_h(\zeta)$ for unstable ($\zeta < 0$) and stable ($\zeta > 0$) conditions - Iterative Obukhov length L solver via `jax.lax.fori_loop` (fully JAX-differentiable) - NaN-safe branching: `jnp.minimum(zeta, -1e-10) / jnp.maximum(zeta, 1e-10)` pattern for gradient safety - Python `if` on string `scheme` parameter resolved at JAX trace time (each scheme traces to a separate computational graph)

Integration across tiles: - `CouplerConfig`, `SurfaceLayerConfig`, `BulkFormulaConfig`, `LandConfig`, `SeaIceConfig`, `LakeConfig` all have `bulk_scheme`, `z_ref`, `bulk_n_iter` fields - Ocean: wind relative to ocean surface current ($u_{\text{rel}} = u_{\text{atm}} - u_{\text{ocean}}$) - Land: moisture-adjusted surface humidity via soil moisture availability factor - Sea ice / Lake: saturated surface humidity at surface temperature

4.5.4 Conservation in Coupling

The coupler enforces that fluxes are conservative: - Heat flux leaving atmosphere = heat flux entering ocean + land + lake + ice - Freshwater is conserved across the atmosphere-surface interface - Momentum transfer is balanced

4.6 Forcing

4.6.1 AMIP Forcing (`forcing/amip.py`)

Prescribed SST and sea-ice boundary conditions for atmosphere-only experiments: - `AMIPForcing` — loads and interpolates monthly SST/sea-ice data - `AMIPForcingConfig` — paths, interpolation method, climatology settings - Supported presets: COBE-SST2, HadISST, custom NetCDF, analytical (Qobs-like)

4.6.2 AMIP Experiment Configuration (`forcing/amip_config.py`)

`AMIPExperimentConfig(NamedTuple)` — complete experiment parameter set: - Grid/integration: resolution, nlev, dt, start_day, days, diag_days, checkpoint_days - Forcing: dataset preset, forcing path, SST/SIC variable names and offsets - Physics: radiation scheme, hyperdiffusion, bulk aerodynamic coefficients, Rayleigh friction - RRTMG: CO₂, CH₄, N₂O concentrations - Surface: emissivity, albedo (ocean/ice), sea ice temperature

Utilities: `config_to_dict()`, `config_from_dict()`, `save_config()`, `load_config()`, `save_checkpoint()`, `load_checkpoint()`.

4.6.3 External Forcing (`forcing/external.py`)

Modular external forcing framework for prescribed boundary conditions:

Config	Parameters	Status
<code>GHGConfig</code>	CO ₂ (348 ppmv), CH ₄ (1650 ppbv), N ₂ O (306 ppbv)	Active (constant + file)
<code>OzoneConfig</code>	Path, variable name, enabled flag	Active (monthly zonal-mean from file)
<code>AerosolConfig</code>	Path, variable name, enabled flag	Active (monthly zonal-mean from file)
<code>SolarConfig</code>	TSI (1360 W/m ²)	Active (constant + file)

Each supports `source="constant"` (default) or `source="file"` (time-varying from NetCDF). Combined via `ExternalForcingConfig`.

File-based interpolation (v3.4): - `_load_nc_timeseries(path, varnames)`: LRU-cached NetCDF loading for 1-D time series (GHG, TSI). Linear interpolation with edge clamping. - `_load_nc_monthly_zonal(path, varname)`: LRU-cached loading for monthly zonal-mean fields (ozone, aerosol). Cyclic interpolation with period 365.25 days. - GHG file format: `time` dimension + `co2_ppmv`, `ch4_ppbv`, `n2o_ppbv` variables. - TSI file format: `time` dimension + `tsi` variable. - Ozone/aerosol file format: `time` (12 months) × `lat` dimensions + `ozone / aod` variable.

Note: Ozone and aerosol data are loaded and interpolated but not yet connected to the radiation solver. This is the primary remaining gap for CMIP forcing readiness. See `docs/cmip_readiness.md`.

4.6.4 Experiment Templates (`forcing/experiments.py`)

CMIP6-style experiment configurations with built-in GHG time series:

Template	Period	Forcing	Parent
piControl	1850–∞	Fixed 1850 (CO ₂ =284.3, CH ₄ =808, N ₂ O=273)	—
historical	1850–2014	Transient (interpolated from key benchmark years)	piControl
ssp245	2015–2100	SSP2-4.5 pathway (CO ₂ reaches ~600 ppmv by 2100)	historical
ssp585	2015–2100	SSP5-8.5 pathway (CO ₂ reaches ~1135 ppmv by 2100)	historical
amip	1979–2014	Prescribed SST/SIC + transient GHG	—
1pctCO2	Year 1–150	1% per year CO ₂ increase from 284.3 ppmv	piControl

Key functions: - `ghg_at_year(experiment_name, year) → (co2_ppmv, ch4_ppbv, n2o_ppbv)` with linear interpolation - `get_ghg_for_experiment(name, year) → dict` version - `create_experiment_config(name, **overrides) → AMIPExperimentConfig` from template

4.6.4 AMIP Spectral Experiment (`scripts/run_amip_spectral.py`)

Full AMIP simulation on the spectral PE dycore with operator-split physics:

Parameter	Value
Dycore	Spectral PE (Gaussian grid, vorticity-divergence)
Resolution	T21/L20 (~5.6°, 20 sigma levels) or T42/L20 (~2.8°)
Time step	600 s (T21), 300 s (T42)
Radiation	Gray (Frierson 2006), moisture-dependent LW OD
Convection	SBM (Frierson 2007, $\tau_c = 2$ hr)
BL exchange	Bulk aerodynamic ($C_H = C_E = 0.0044$)
Condensation	Saturation adjustment
Friction	Rayleigh (BL drag + free-atmosphere)
Hyperdiffusion	∇^4 , 0.5 hr e-folding at n_{max}
Forcing	Analytical (Qobs-like SST + seasonal cycle) or NetCDF

Physics coupling: Gray radiation and Rayleigh friction are coupled inside the RK stages (spectral tendencies). BL exchange, SBM convection, and condensation are operator-split after each dynamics step; the updated grid-space temperature is transformed back to spectral coefficients.

Completed run (v3.1): 365-day T21/L20 simulation with analytical forcing, gray radiation, and fixed CO₂ (415 ppmv). Key diagnostics at equilibrium:

Dagnostic	Day 365
<T_atm>	266.1 K

Diagnostic Day 365	
<T_low>	288.4 K
<Precip>	4.6 mm/day
<CWV>	43.1 kg/m ²
max\ v\	49.6 m/s
LW_TOA	254.8 W/m ²
Wall time	42 min (CPU, Apple M-series)

Clear seasonal cycle in all diagnostics; zonal-mean structure with ITCZ precipitation, midlatitude jets, and realistic radiative balance.

4.6.5 AMIP FV Cubed-Sphere Experiment (`scripts/run_amip.py`)

Full AMIP simulation on the FV cubed-sphere dycore with operator-split physics:

Parameter	Value
Dycore	Hydrostatic PE on cubed-sphere (centered or FV)
Resolution	C16/L20 to C48/L40
Vertical coordinate	Sigma or hybrid sigma-pressure (selectable)
Time step	450–600 s
Radiation	Selectable: gray (Frierson 2006) or RRTMGP (correlated-k)
Diurnal cycle	Optional instantaneous solar zenith angle
Ozone	Standard (US Std Atm 1976), analytical (lat-dependent), or none
Clouds	None (clear-sky), Sundqvist (RH-based), or Xu-Randall (RH+condensate)
Convection	SBM (Frierson 2007, $\tau_c = 2$ hr)
Microphysics	None, Kessler, Sundqvist, Seifert-Beheng, Morrison, Thompson
BL exchange	Bulk aerodynamic (constant, COARE3, or Large-Yeager)
Surface albedo	Constant or dynamic (snow feedback + ice T-dependent + ocean zenith)
Condensation	Saturation adjustment (disabled when microphysics active)
Friction	Rayleigh (BL drag + free-atmosphere)
Topography	Flat, Gaussian mountain, or real (ETOPO1 NetCDF)
Forcing	COBE-SST2, HadISST, analytical, or custom SST

Key features: - Checkpoint/restart via NPZ format with embedded JSON config - Radiation cadence: `--rad-update-steps N` (hold tendencies between calls) - RRTMGP with interactive H₂O and prescribed CO₂/CH₄/N₂O/O₃ - Energy budget tracking (R_TOA, column energy, dE/dt, residual) - Monthly-mean zonal diagnostics via `--monthly-means` - `AMIPExperimentConfig` for full experiment reproducibility

Completed run (v3.2): 365-day C16/L20 simulation with gray radiation:

Diagnostic	Day 365
LW_TOA	236 W/m ²
<Precip>	4 mm/day

Production configuration (v3.3): 10-year C48/L40 AMIP (`scripts/run_amip_production.sh`) with RRTMGP, diurnal cycle, analytical ozone, Xu-Randall clouds, Kessler microphysics, dynamic surface albedo, monthly checkpointing, and auto-validation via `scripts/validate_amip.py` .

Validation targets (v3.3): T_{2m}: 287–289 K, precipitation: 2.5–3.0 mm/day, TOA imbalance: < 1 W/m², OLR: 235–245 W/m², subtropical jet: 30–40 m/s at ~30° lat.

See `docs/amip.md` for full CLI reference, radiation modes, and diagnostics.

4.7 Machine Learning Module (`ml/`)

4.7.1 SFNO Architecture

Spherical Fourier Neural Operator for learned dynamics: - `sfno.py` — Main SFNO class (Equinox module) - `sfno_block.py` — SFNO block: spectral convolution + MLP - `spectral_conv.py` — Spherical spectral convolution layer - `normalization.py` — Channel normalization

4.7.2 State Packing (`ml/channel_packing.py`)

Converts model state (SW/PE/Ocean) to/from ML channel format for SFNO input/output.

4.7.3 Conservation Correction (`ml/conservation.py`)

Post-hoc conservation filters for ML predictions: - Mass fixer (uniform additive correction) - Moisture fixer (column-wise) - Heat conservation - Salt conservation (ocean)

4.7.4 Training (`ml/training.py` , `ml/loss.py`)

- Differentiable rollout training via `jax.grad` through `lax.scan`
- Loss functions: MSE, weighted MSE, spectral loss
- Optimizer integration via Optax

4.7.5 Data Loading (`ml/data/era5_loader.py`)

ERA5 reanalysis data loader for training and initialization.

5. Differentiability Design

5.1 End-to-End Gradient Flow

The entire model is a composition of differentiable functions:

```
loss = L(observe(integrate(initialize(params), n_steps)), observations)
grad_loss = jax.grad(loss)(params)
```

where `params` can include: - Physical parameters (diffusion coefficients, drag coefficients, etc.) - Neural network weights (for ML parameterizations) - Initial conditions (for 4D-Var data assimilation)

5.2 Gradient Computation Strategy

Technique	Purpose
<code>jax.grad</code>	Reverse-mode AD for parameter gradients
<code>jax.jvp</code>	Forward-mode for tangent linear model
<code>jax.checkpoint</code>	Rematerialization to reduce memory for long rollouts
<code>jax.lax.scan</code>	Efficient sequential time integration with automatic checkpointing
Gradient clipping	Prevent exploding gradients in long rollouts
Truncated BPTT	Optional: backprop through fixed windows (e.g., 5 days)

5.3 Checkpointing Strategy

For a 10-day forecast at 15-min timesteps = 960 steps. Without checkpointing: must store all 960 intermediate states. With checkpointing: store every Kth state, recompute the rest.

```
def integrate(state, n_steps, dt, model):
    def scan_fn(state, _):
        state = jax.checkpoint(model.step)(state, dt)
        return state, state
    final_state, trajectory = jax.lax.scan(scan_fn, state, jnp.arange(n_steps))
    return final_state, trajectory
```

5.4 4D-Var Data Assimilation (Planned)

```
def cost_4dvar(x0, observations, model, B_inv, R_inv):
    """4D-Var cost function – differentiable end-to-end."""
    J_b = 0.5 * (x0 - x_b).T @ B_inv @ (x0 - x_b)

    trajectory = model.integrate(x0, n_steps)
    J_o = 0.0
    for obs in observations:
        H_x = obs.operator(trajectory[obs.time_index])
        d = obs.values - H_x
        J_o += 0.5 * d.T @ R_inv @ d

    return J_b + J_o

grad_J = jax.grad(cost_4dvar)
x0_optimal = optimize(cost_4dvar, grad_J, x0_initial)
```

6. Hardware & Parallelism

6.1 Target Platforms

Platform	Priority	Backend	Notes
NVIDIA GPU (single)	P0	CUDA via JAX	Development & testing
NVIDIA GPU (multi, single node)	P0	JAX sharding	Primary production target
Apple M-series (Metal/MPS)	P1	jax-metal	Development on M4 MacBook
CPU (multi-core)	P1	JAX default	Fallback, CI/CD
NVIDIA GPU (multi-node)	P2	mpi4jax	Large-scale production
Google TPU	P2	JAX native	Cloud scaling

6.2 Parallelism Strategy

6.2.1 Single-Node: JAX Device Mesh (`parallel/mesh.py`)

Face-dimension sharding across 1–6 devices:

```
config = create_device_mesh(n_devices="auto") # auto-detects, clamps to 1/2/3/6
```

- `DeviceConfig(mesh, face_sharding, replicated_sharding, n_devices, backend, is_distributed)`
- Single device → `mesh=None`, no sharding overhead
- Multi-device → `Mesh(devices, ("face",))`, `PartitionSpec("face")` on first axis
- `shard_pytree(state, config)` — shards `(6,n,n,...)` arrays by face dimension
- `replicate_pytree(data, config)` — replicates data across all devices

6.2.2 Multi-Node: MPI via `mpi4jax` (`parallel/distributed.py`, `parallel/halo_exchange.py`)

```
config, topology = initialize_distributed(return_topology=True)
```

CommTopology (`parallel/comm.py`): - Deterministic face-to-rank mapping: valid for 1, 2, 3, or 6 MPI ranks - `neighbor_ranks[(face, edge)]` — which rank owns each neighbor - Packed edge exchange to minimize MPI messages

MPI Halo Exchange (`parallel/halo_exchange.py`): - Canonical edge ordering for packed correctness - Local edges handled without MPI (direct array indexing)

Global Reductions (`parallel/reductions.py`): - MPI-aware global sums, means, and integrals

6.2.3 Voronoi Mesh Decomposition (`parallel/voronoi_partition.py`, `parallel/halo_exchange_voronoi.py`)

Domain decomposition for unstructured MPAS/Voronoi C-grid meshes:

```

from legoesm.parallel import partition_voronoi_mesh, build_local_mesh,
VoronoiHaloExchange

# Partition mesh across 4 MPI ranks
part = partition_voronoi_mesh(mesh, n_ranks=4, rank=my_rank, halo_depth=2)
local_mesh = build_local_mesh(mesh, part)

# Exchange halo data
halo = VoronoiHaloExchange(part, backend="mpi")
u_local = halo.exchange_edge_field(u_local)

```

Partitioning strategies: - `partition_cells_geometric()` — Recursive Coordinate Bisection (RCB) using 3D cell centers. No external dependencies. - `partition_cells_metis()` — k-way graph partitioning via pymetis. Better load balancing for variable-resolution meshes.

Data structures: - `VoronoiPartition` — owned/halo cell/edge/vertex lists, global sizes, communication schedules per entity type. - `HaloCommSchedule` — neighbor ranks, send/rcv counts, send/rcv index arrays (sorted by global index for deterministic matching).

Entity ownership: edges owned by rank of `cell_owner[min(cellsOnEdge)]`; vertices by `cell_owner[min(cellsOnVertex)]`.

Halo exchange: MPI sendrecv with entity-type tag offsets (cell=0, edge=1M, vertex=2M). Local mesh construction remaps connectivity to local indices; non-local entries → -1 (masked by TRiSK operators via `(idx >= 0)`).

6.2.4 Ensemble Parallelism (`parallel/ensemble.py`)

Run large ensembles (10–100+ members) of independent simulations:

```

from legoesm.parallel.ensemble import (
    perturb_initial_conditions, make_ensemble_step,
    ensemble_integrate, ensemble_mean, shard_ensemble,
)

# Create 32 perturbed initial conditions
batched = perturb_initial_conditions(state, key, n_members=32, scale=0.01)

# Wrap model step with vmap (grid/config shared, state batched)
ensemble_step = make_ensemble_step(model.step, in_axes=(0, None))

# Time-integrate all members (scan outside, vmap inside)
final = ensemble_integrate(ensemble_step, batched, n_steps=1000, dt=600.0)

# Multi-device: shard ensemble across GPUs
mesh = create_ensemble_mesh(n_members=64)
sharded = shard_ensemble(batched, mesh)

```

Strategies: (a) `vmap` — single device, XLA fuses kernels; (b) `sharded` — `NamedSharding` on ensemble axis; (c) `hybrid` — shard across devices + `vmap` within.

Features: `jax.checkpoint` for $O(\sqrt{n})$ memory in AD, `ensemble_integrate_with_forcing()` for time-varying forcing, `perturb_parameters()` for parameter perturbation experiments.

6.3 Mixed Precision

```
# Dynamics: float32 for accuracy
dynamics_dtype = jnp.float32

# ML parameterizations: bfloat16 for speed
ml_dtype = jnp.bfloat16

# Conservation fixer: float64 for global sums
conservation_dtype = jnp.float64
```

6.4 Apple Silicon Support (parallel/metal.py , core/hardware.py)

Backend detection: detect_devices() returns backend, n_devices, supports_f64, distributed status.
Backend priority: METAL → GPU → TPU → CPU.

Metal limitations: - No float64 or complex128 support - Spectral transforms automatically routed to CPU device - get_metal_config() → MetalConfig(is_metal, metal_device, cpu_device)

7. I/O & Data Pipeline

7.1 Primary Format: Zarr

```
def save_state(state, path, time_index):
    ds = state_to_xarray(state)
    ds.to_zarr(path, mode='a', append_dim='time')

def load_state(path, time_index=-1):
    ds = xr.open_zarr(path)
    return xarray_to_state(ds.isel(time=time_index))
```

7.2 ERA5 Initialization (ml/data/era5_loader.py)

ERA5 reanalysis data loading for model initialization and ML training.

7.3 Online Diagnostics (diagnostics/)

Computed during runtime at configurable intervals:

Diagnostic	Variables	Level
Surface maps	T_2m, u_10m, v_10m, MSLP, precipitation	Surface
Pressure-level maps	Z, T, q, u, v	500 hPa, 850 hPa
Global means	Total energy, total mass, total moisture	Scalar
Conservation budget	Energy/mass/moisture residuals per timestep	Scalar
Spectra	Kinetic energy spectrum vs. wavenumber	Global

Energy Budget Closure (`diagnostics/energy_budget.py`)

Column-integrated moist static energy: $E = \int (c_p T + L_v q + \Phi + \frac{1}{2}v^2) dp/g$, computed via hydrostatic geopotential integration with `jax.lax.scan`. `EnergyBudgetTracker` accumulates timeseries of `R_TOA`, column energy, `dE/dt`, and residual (`R_TOA - dE/dt`). Also provides `toa_net_radiation()`, `surface_net_radiation()`, `surface_energy_flux()` utility functions.

Monthly Means (`diagnostics/monthly_means.py`)

`MonthlyAccumulator` for long climate runs (10-year AMIP): - Bins 2D cubed-sphere fields into zonal-mean latitude bands (configurable `n_lat_bins`) - Accumulates 3D fields as zonal-mean vertical profiles (`n_lat × nlev`) - Tracks global-mean scalars per calendar month (365-day calendar) - `finalize()` computes monthly averages, `save()` writes to NPZ - Activated via `--monthly-means` CLI flag in `run_amip.py`

7.4 CF/CMOR-Compliant Output (`io/cmor_output.py`)

`CFWriter` class for CMIP6-class NetCDF output:

- **CF-1.8 conventions:** `standard_name`, `long_name`, `units`, `cell_methods`, `time bounds`
- **CMIP6 DRS naming:** `<var>_<table>_<model>_<expt>_<variant>_<grid>[_<trange>].nc`
- **Two CMOR tables:** `Amon` (21 atmospheric monthly variables) and `Lmon` (6 land monthly variables)
- **Standard pressure grid:** `CMIP6_PLEV19` — 19 standard pressure levels
- **Variable lookup:** `lookup_cmor_entry(var_name)` → (`table_id`, `metadata_dict`)
- **Context manager** and explicit `close()` support
- **Lazy imports:** `xarray` and `netCDF4` imported only when writing (not at module load)

```
writer = CFWriter(output_dir="output/", experiment_id="historical",
model_id="legoESM-1-0", freq="mon")
writer.write_field("tas", data, time=15.0, time_bounds=(0.0, 30.0), lat=lat, lon=lon)
writer.write_monthly(monthly_data, lat, lon, plev=CMIP6_PLEV19) # from
MonthlyAccumulator
writer.close()
```

7.5 Restart and Reproducibility (`io/restart.py`)

Enhanced restart I/O with cryptographic integrity verification:

- **RestartMetadata** (NamedTuple, 13 fields): `model_version`, `creation_time`, `platform`, `jax_version`, `jax_x64_enabled`, `numpy_version`, `config_hash`, `state_digest`, `resolution`, `nlev`, `step`, `day`, `git_hash`
- **save_restart()** : Writes `.npz` checkpoint + `.meta.json` sidecar with SHA-256 state digest and config hash
- **load_restart()** : Loads checkpoint with optional strict validation (`resolution`, `nlev`, `config hash`, `state digest match`)
- **verify_reproducibility(path_a, path_b)** → `ReproducibilityReport(identical, differences, state_digest_match, config_match)`
- **compute_state_digest()** : SHA-256 of sorted, concatenated array bytes (deterministic across runs)
- **compute_config_hash()** : SHA-256 of JSON-serialized config

7.6 Tuning Guide (`tuning.py`)

Executable tuning documentation with parameter registry and validation:

- `TUNING_PARAMETERS` : Registry of 16 parameters across 5 categories (dynamics, radiation, convection, diffusion, surface) with valid ranges, default values, sensitivity ratings (high/medium/low), and physical notes
- `validate_tuning(config)` : Checks an `AMIPExperimentConfig` for problematic settings (CFL violations, range exceedances, conflicting options)
- `recommended_params(resolution, nlev)` : Returns resolution-appropriate defaults (`dt`, `hyperdiff_scale`, `rad_update_steps`) with CFL-based scaling
- `print_tuning_guide()` : Formatted table of all parameters by category

7.7 Visualization (`visualization/maps.py`)

Global field plotting with Cartopy (Mollweide projection) and conservation timeseries.

8. Software Engineering

8.1 Language & Dependencies

Dependency	Version	Purpose
Python	>= 3.11	Runtime
JAX	>= 0.4.35 (tested 0.8–0.9)	Core compute framework
jaxlib	matching JAX	XLA backends
jax-metal	latest	Apple Silicon GPU support (FV solvers only, no float64)
Equinox	>= 0.11	Neural network modules (pytree-based)
Optax	>= 0.2	Optimizers for training/DA
xarray	>= 2024.0	Data structures for I/O
zarr	>= 2.18	Storage format
matplotlib	>= 3.9	Visualization
cartopy	>= 0.23	Map projections
mpi4py	>= 4.1, < 5	MPI bindings (optional)
mpi4jax	>= 0.8, < 0.9	Multi-node parallelism (optional)
pytest	>= 8.0	Testing
pyyaml	>= 6.0	Configuration
netCDF4	>= 1.6	NetCDF file support

8.2 Dependency Management

hatchling build system with `pyproject.toml`:

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "legoesm"
version = "0.1.0"
requires-python = ">=3.11"

[project.optional-dependencies]
data = ["gcsfs", "fsspec"]
metal = ["jax-metal"]
mpi = ["mpi4py>=4.1,<5", "mpi4jax>=0.8,<0.9"]
dev = ["pytest>=8.0", "ruff", "mypy"]

[project.scripts]
legoesm = "legoesm.cli:main"
```

8.3 Code Quality

- **Formatter:** `ruff format`
- **Lint:** `ruff check`
- **Type checking:** `mypy`
- **CI/CD:** GitHub Actions (MPI distributed tests, nightly integration)

9. API Design

9.1 Python API

```
import legoesm as desm

# ---- Quick start: Create and run a model ----
grid = desm.grids.CubedSphere(resolution="C48", n_levels=37)
atmosphere = desm.atmosphere.DynamicalCore(
    grid=grid,
    equations="hydrostatic",
    discretization="finite_volume",
)

# Initialize
state = desm.atmosphere.physics.baroclinic_wave.initialize(grid, sigma_coord)

# Run a 10-day integration
trajectory = atmosphere.integrate(state, duration=desm.days(10), dt=desm.minutes(15))

# ---- Differentiable: compute gradients ----
def forecast_loss(params, state):
    atm = desm.atmosphere.DynamicalCore(grid=grid, params=params)
    trajectory = atm.integrate_scan(state, n_steps=100, dt=900.0)
    return jnp.mean(trajectory.T ** 2)

grads = jax.grad(forecast_loss)(params, state)
```

```
# ---- Coupled model ----  
coupler = desm.coupler.make_coupler(config)
```

9.2 YAML Configuration

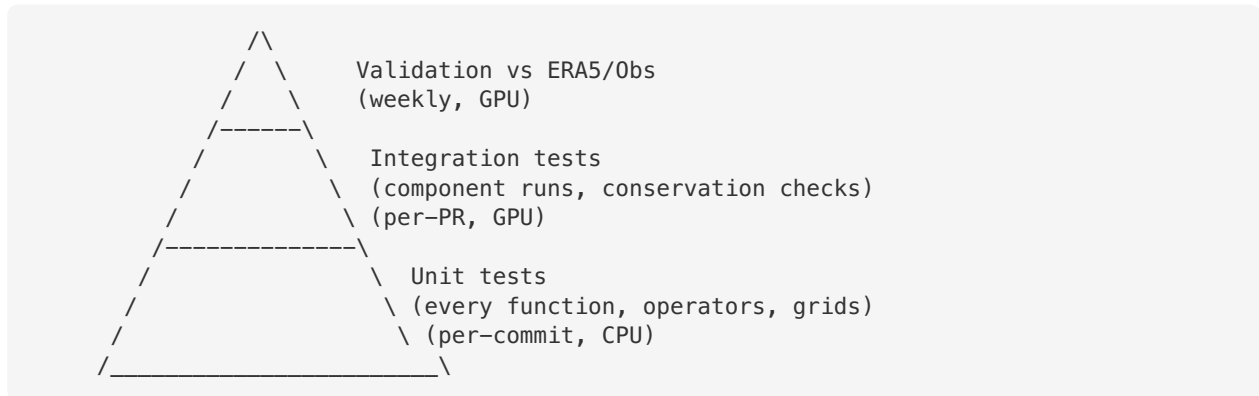
```
model:  
  name: "legoESM weather forecast"  
  type: coupled  
  
grid:  
  type: cubed_sphere  
  resolution: C48  
  n_levels: 20  
  vertical_coord: sigma  
  
atmosphere:  
  equations: hydrostatic  
  discretization: finite_volume  
  dt_seconds: 900  
  
physics:  
  radiation:  
    type: gray  
  convection:  
    type: sbm  
  microphysics:  
    type: kessler  
  turbulence:  
    type: louis  
  gravity_wave_drag:  
    type: rayleigh  
  
ocean:  
  type: slab  
  depth: 50.0  
  
coupler:  
  coupling_dt_seconds: 1800  
  tiles: [ocean, land, lake, ice]
```

9.3 Command-Line Interface (cli.py)

```
# Run a simulation  
legoesm run config/weather_forecast.yaml  
  
# Run Williamson test case  
legoesm test williamson --case 2 --resolution C48 --days 5  
  
# Benchmark performance  
legoesm benchmark --grid C384 --n-steps 100 --devices 4
```

10. Testing & Validation

10.1 Testing Pyramid



10.2 Test Suite (85+ test files, 1600+ tests)

Unit tests (`tests/unit/`):

Category	Tests	Description
Core	<code>test_grid.py</code> , <code>test_latlon_grid.py</code> , <code>test_halo.py</code> , <code>test_operators.py</code> , <code>test_operators_fv.py</code> , <code>test_operators_latlon.py</code> , <code>test_operators_fc.py</code> , <code>test_operators_cgrid.py</code> , <code>test_cgrid_metrics.py</code> , <code>test_field.py</code> , <code>test_conservation.py</code> , <code>test_smooth.py</code>	Grid creation, operator correctness, conservation fixers
Dynamics	<code>test_shallow_water_fv.py</code> , <code>test_shallow_water_fv_latlon.py</code> , <code>test_shallow_water_fc.py</code> , <code>test_shallow_water_cgrid.py</code> , <code>test_primitive_eq.py</code> , <code>test_primitive_eq_fv_latlon.py</code> , <code>test_primitive_eq_fc.py</code> , <code>test_compressible_euler.py</code> , <code>test_compressible_euler_fc.py</code> , <code>test_spectral.py</code> , <code>test_spectral_pe.py</code> , <code>test_spectral_nh.py</code> , <code>test_fc_gram.py</code>	Model stepping, conservation, differentiability
Physics	<code>test_radiation.py</code> , <code>test_convection.py</code> , <code>test_turbulence.py</code> , <code>test_microphysics.py</code> , <code>test_gravity_wave_drag.py</code> , <code>test_combined_physics.py</code> , <code>test_all_physics_schemes.py</code>	Individual scheme correctness, factory pattern
Ocean	<code>test_ocean.py</code> , <code>test_ocean_compatibility.py</code> , <code>test_ocean_fc.py</code>	EOS, z-star, stepping, conservation
ML	<code>test_sfno.py</code> , <code>test_sfno_sw.py</code> , <code>test_sfno_ocean.py</code>	SFNO architecture, channel packing
Coupled	<code>test_coupler.py</code> , <code>test_atmosphere_invariants.py</code> , <code>test_external_forcing.py</code>	Tile blending, flux conservation, MOST bulk fluxes, ocean albedo plumbing, q_surface consistency, external forcing file interpolation

Category	Tests	Description
Infrastructure	<code>test_timestepping.py</code> , <code>test_thermodynamics.py</code> , <code>test_tracer_transport.py</code> , <code>test_backend_guard.py</code> , <code>test_backend_precision.py</code> , <code>test_parallel.py</code> , <code>test_ensemble.py</code>	Time integrators, thermodynamics, parallelism, ensemble vmap/sharding
Land	<code>test_multilayer_land.py</code> , <code>test_carbon_cycle.py</code> , <code>test_stomata.py</code> , <code>test_surface_albedo.py</code> , <code>test_topography.py</code>	Carbon pools, stomatal conductance, soil hydraulics, albedo
CMIP infra	<code>test_cmor_experiments_restart.py</code>	CMOR tables, CFWriter, experiment templates, GHG interpolation, restart digests, tuning validation
Validation	<code>test_weatherbench.py</code> , <code>test_dcmip_transport.py</code> , <code>test_issue_fixes.py</code> , <code>test_amip_config.py</code>	WeatherBench metrics, DCMIP transport, AMIP config

Integration tests (`tests/integration/`): - `test_shallow_water.py` — Full SW integration - `test_amip_smoke.py` — AMIP simulation smoke test - `test_amip_rrtmg.py` — AMIP with RRTMGP radiation (8 tests) - `test_fv_cubesphere.py` — FV cubed-sphere integration

Distributed tests (`tests/distributed/`): - `test_halo_mpi.py`, `test_coupler_mpi.py`, `test_ocean_mpi_conservation.py` - `test_voronoi_halo.py` — Voronoi mesh partitioning, local mesh, operator equivalence, halo exchange (34 tests)

Land stability tests (`tests/land/`): - `test_land_stability.py` — Multi-day slab & multi-layer land integrations with realistic forcing (32 tests): - `TestSlabLandStability` (6): 30-day run — T bounds, W bounds, snow, diurnal cycle - `TestMultiLayerStability` (7): 15-day Richards+thermal — T/θ bounds, ψ finite, runoff, deep vs. surface - `TestSlabEnergyBudget` (1): 10-day energy closure <5% - `TestSlabWaterBudget` (1): 10-day water balance - `TestSnowCycle` (3): accumulation, melt, albedo feedback - `TestCarbonCycleIntegration` (4): 30-day DifferLand — pools positive, evolving, hierarchy - `TestMultiLayerPhysics` (4): smooth T profile, physical θ, deep stability - `TestMultiLayerCarbon` (2): multi-layer + carbon coupling - `TestSeasonalBehavior` (4): 180-day Jan→Jun — midlat/polar warming, no runaway, seasonal range

Validation tests (`tests/validation/`): - Bulk flux differentiability and all-tile tests - Ocean model differentiability across all 4 discretizations (`test_differentiability_ocean.py`, `pytest-parametrized`) - Spectral PE stability analysis (E-variable, eigenvalues) - Held-Suarez fix verification - Dycore progression suite

Test cases (`tests/test_cases/`): - Williamson TC2/TC5 on cubed-sphere and lat-lon - DCMIP transport tests - DCMIP-2025 test cases (TC1, TC2, TC3)

10.3 Williamson Test Cases

Test	Description	Validation Criteria
Test 2	Steady-state nonlinear zonal geostrophic flow	Height field error < 1e-5 after 5 days
Test 5	Zonal flow over isolated mountain	Conservation: mass < 1e-12, energy < 1e-8

10.4 Held-Suarez Test

- 1200-day integration with Newtonian relaxation + Rayleigh friction
- Validate: zonal-mean zonal wind (jet structure), temperature, eddy statistics
- Implemented for spectral PE, cubed-sphere, and lat-lon grids

10.5 DCMIP Test Cases

- DCMIP transport: 3D deformational flow
- DCMIP-2025: TC1 (baroclinic wave), TC2 (tropical cyclone), TC3 (supercell)

10.6 WeatherBench2 Evaluation (`evaluations/`)

- Evaluation metrics: RMSE, ACC, bias, spread-skill ratio
- Framework for comparing against ERA5 reanalysis
- Configurable via YAML

11. Milestone Roadmap (Detailed)

Milestone 1: Dynamical Core MVP — COMPLETE

Delivered: - Shallow-water equations on cubed-sphere (centered + FV) - Williamson Test 2 and 5 passing - `jax.grad` through rollout producing finite gradients - Conservation: mass to machine precision, energy to $< 1e-8$ - Runs on CPU, NVIDIA GPU, and Apple M-series - Visualization, YAML config, CLI

Milestone 2: 3D Primitive Equations — COMPLETE

Delivered: - Hydrostatic PE with sigma vertical coordinate (cubed-sphere, lat-lon, spectral) - Non-hydrostatic compressible Euler with z^* coordinate and split-explicit time stepping - Held-Suarez test case validation (spectral + cubed-sphere + lat-lon) - Baroclinic wave initialization - Multi-GPU sharding (single node, face-dimension)

Milestone 3: Physics Parameterizations — COMPLETE

Delivered: - Standard physics factory pattern (config $\rightarrow$ integration $\rightarrow$ physics_fn) - **Radiation:** Gray (Frierson 2006), RRTMGP (full correlated-k) - **Convection:** SBM, DCA, Kuo, mass-flux (Arakawa-Wu), EDMF (5 schemes) - **Microphysics:** Kessler, Sundqvist, Seifert-Beheng, Morrison, Thompson, ML emulator (6 schemes) - **Turbulence:** Smagorinsky, Louis, TKE/MY2.5, CLUBB-lite, Holtslag-Boville, YSU, EDMF, ML emulator (8 schemes) - **Gravity wave drag:** Rayleigh, Lindzen, McFarlane, Hines, prognostic spectral, ML emulator (6 schemes) - Combined physics suite orchestrator - Thermodynamic utilities

Milestone 4: ML-Driven Parameterization — COMPLETE

Delivered: - SFNO (Spherical Fourier Neural Operator) architecture in Equinox - Learned dynamics: SFNO SW, SFNO PE, SFNO Ocean - Channel packing for state $\rightarrow$ ML channels $\rightarrow$ state - Conservation correction filters for ML predictions - Differentiable rollout training with loss functions - ERA5 data loader

Milestone 5: Ocean + Land + Ice + Coupler — COMPLETE

Delivered: - **Ocean:** Full 3D PE with z-star coordinates, Wright EOS, split-explicit barotropic, spectral variant, SFNO variant, comprehensive physics (vertical mixing: constant/Richardson/KPP; lateral mixing: harmonic/biharmonic/GM-Redi; surface forcing: prescribed/restoring/bulk; bottom drag: linear/quadratic; convection: enhanced diffusion/plume) - **Land:** Slab + multi-layer land, Richards equation, 6 retention curves, carbon cycle (DALEC-990 6-pool + seasonal), Farquhar photosynthesis + Ball-Berry/Medlyn/Jarvis stomata - **Sea ice:** Thermodynamic slab + EVP rheology (Hunke & Dukowicz 1997) + multi-category (Lipscomb 2001) - **Lake:** Two-layer lake model - **Coupler:** Tile-based surface exchange with flux accumulation, conservation enforcement - **AMIP forcing:** Prescribed SST/sea-ice boundary conditions - Simple/slab ocean for atmosphere-only experiments

Milestone 6: Lat-Lon Grid + Finite-Volume Transport — COMPLETE

Delivered: - Lat-lon grid with halo exchange (periodic lon, zero-grad poles) - Centered and FV (PPM) operators for lat-lon - All three equation sets on lat-lon: SW, PE, CE (both centered and FV) - Polar filter (Fourier-based CFL stabilization) - Williamson test cases on lat-lon - Held-Suarez on lat-lon - Key A-grid energy consistency insight documented

Milestone 7: Full Physics Suite — COMPLETE

Delivered: - 25+ physics parameterizations across 5 categories - ML emulator variants for microphysics, turbulence, and GWD - RRTMGP full radiation (JAX port) - Solar geometry utilities - All schemes follow standard factory pattern and support hydrostatic/non-hydrostatic/spectral

Milestone 8: Data Assimilation — PLANNED

Goals: - 4D-Var cost function (differentiable end-to-end) - Adjoint model via `jax.grad` - Observation operators (radiance, in-situ) - Assimilation cycling experiments - Ensemble Kalman filter variant

Milestone 9: Climate-Scale Simulations — IN PROGRESS

Delivered: - 365-day AMIP spectral simulation (T21/L20, gray radiation, SBM convection, analytical SST forcing) - 365-day AMIP FV cubed-sphere simulation (C16/L20, gray radiation, SBM convection) - Seasonal cycle reproduced with physically realistic diagnostics - Critical bug fixes enabling year-long stability: E-variable PGF correction, SBM cloud-layer masking - Unified MOST bulk flux module across all surface tiles (ocean, land, sea ice, lake) - FC-Gram high-order dynamical cores (6 models) with optional divergence damping - C-grid variants: true C-grid on lat-lon, divergence-damped on cubed-sphere (4 models) - External forcing framework (GHG, ozone, aerosol, solar) - Checkpoint/restart for AMIP experiments - Land carbon cycle (DALEC-990 6-pool + seasonal scheme) - Plant physiology (Farquhar + Ball-Berry/Medlyn + Jarvis stomata) - Ocean biogeochemistry (abiotic DIC/ALK + NPZD ecosystem) - Sea-ice dynamics (EVP rheology + multi-category ice) - CMOR/CF-compliant output pipeline (27 variables, CMIP6 DRS naming) - Experiment templates (piControl, historical, SSP2-4.5, SSP5-8.5, AMIP, 1pctCO₂) - Restart/reproducibility with SHA-256 state digests - Documented tuning guide (16 parameters, validation, recommended defaults)

Remaining goals: - Multi-century stability testing - Higher resolution (T42+) year-long simulations - Ice sheet dynamics - Multi-node scaling validation (mpi4jax at scale) - Full ESM configuration (atmosphere + ocean + land + ice + carbon all coupled) - Climate scenario simulations (SSP2-4.5, SSP5-8.5) with

12. Repository Structure (Actual)

```

LegoESM/
├── pyproject.toml                # Project metadata, dependencies
├── SPECIFICATION.md              # This document
├── README.md                     # Quick start and feature overview
├── .github/workflows/           # CI/CD (MPI distributed, nightly)
├── src/legoesm/
│   ├── __init__.py              # Public API, time conversion helpers
│   ├── cli.py                   # Command-line interface
│   ├── config.py                # YAML config loading + validation
│   ├── constants.py             # Physical constants
│   ├── core/
│   │   ├── field.py             # Core infrastructure
│   │   │   ├── field.py         # Field dataclass (JAX pytree leaf)
│   │   │   ├── state.py        # State containers
│   │   │   ├── operators.py     # 2D centered operators (cubed-sphere)
│   │   │   ├── operators_3d.py  # 3D centered operators (cubed-sphere)
│   │   │   ├── operators_latlon.py # 2D centered operators (lat-lon)
│   │   │   ├── operators_latlon_3d.py # 3D centered operators (lat-lon)
│   │   │   ├── operators_fv.py  # 2D FV/PPM operators (cubed-sphere)
│   │   │   ├── operators_fv_3d.py # 3D FV/PPM operators (cubed-sphere)
│   │   │   ├── operators_fv_latlon.py # 2D FV/PPM operators (lat-lon)
│   │   │   ├── operators_fv_latlon_3d.py # 3D FV/PPM operators (lat-lon)
│   │   │   ├── operators_fc.py  # 2D FC-Gram operators (cubed-sphere)
│   │   │   ├── operators_fc_3d.py # 3D FC-Gram operators (cubed-sphere)
│   │   │   ├── fc_gram.py       # FC-Gram basis and differentiation
│   │   │   ├── conservation.py  # Conservation fixers
│   │   │   ├── smooth.py        # Smooth approximations
│   │   │   └── hardware.py      # Device detection, backend info
│   │   └── grids/
│   │       ├── cubed_sphere.py  # Grid implementations
│   │       │   ├── cubed_sphere.py # CubedSphereGrid (6-face gnomonic)
│   │       │   ├── latlon.py     # LatLonGrid (regular lat-lon)
│   │       │   ├── gaussian.py   # GaussianGrid + SH transforms
│   │       │   ├── vertical.py   # SigmaCoordinate, HeightCoordinate
│   │       │   ├── halo.py       # Cubed-sphere halo exchange
│   │       │   ├── halo_latlon.py # Lat-lon halo exchange
│   │       │   ├── polar_filter.py # Fourier polar filter
│   │       │   ├── topography.py  # Terrain generation
│   │       │   └── regridding.py  # Inter-grid regridding
│   │   └── timestepping/
│   │       ├── ssp_rk3.py        # Time integration
│   │       │   ├── ssp_rk3.py    # SSP-RK3
│   │       │   ├── ssp_rk34.py   # SSP-RK3(4)
│   │       │   ├── ssp_rk54.py   # SSP-RK5(4)
│   │       │   ├── split_explicit.py # Split-explicit RK3
│   │       │   ├── semi_implicit.py # Semi-implicit (spectral PE)
│   │       │   └── tridiagonal.py # Thomas algorithm
│   │   └── atmosphere/
│   │       ├── dynamics/
│   │       │   ├── __init__.py   # Atmosphere component
│   │       │   └── __init__.py   # Factory: make_model(equations,
discretization)
│   │       ├── shallow_water.py  # Centered SW (cubed-sphere)
│   │       │   ├── shallow_water.py # FV SW (cubed-sphere)
│   │       │   └── shallow_water_fv_latlon.py # FV SW (lat-lon)

```

- primitive_eq.py # Centered PE (cubed-sphere)
- primitive_eq_fv.py # FV PE (cubed-sphere)
- primitive_eq_latlon.py # Centered PE (lat-lon)
- primitive_eq_fv_latlon.py # FV PE (lat-lon)
- compressible_euler.py # Centered CE (cubed-sphere)
- compressible_euler_fv.py # FV CE (cubed-sphere)
- compressible_euler_fv_latlon.py # FV CE (lat-lon)
- spectral_sw.py # Spectral SW (Gaussian)
- spectral_pe.py # Spectral PE (Gaussian)
- spectral_nh.py # Spectral NH (Gaussian)
- sfno_sw.py # Learned SW (SFNO)
- sfno_pe.py # Learned PE (SFNO)
- shallow_water_fc.py # FC-Gram SW (cubed-sphere)
- shallow_water_fc_cgrid.py # FC-Gram SW + div damping
- primitive_eq_fc.py # FC-Gram PE (cubed-sphere)
- primitive_eq_fc_cgrid.py # FC-Gram PE + div damping
- compressible_euler_fc.py # FC-Gram CE (cubed-sphere)
- compressible_euler_fc_cgrid.py # FC-Gram CE + div damping
- shallow_water_cgrid_latlon.py # True C-grid SW (lat-lon)
- shallow_water_cgrid.py # A-grid SW + div damping (cubed-sphere)
- primitive_eq_cgrid.py # A-grid PE + div damping (cubed-sphere)
- compressible_euler_cgrid.py # A-grid CE + div damping (cubed-sphere)
- tracer_transport.py # Passive tracer advection
- edge_blending.py # Face boundary blending

physics/

- __init__.py # Physics exports
- combined.py # Physics suite combiner
- thermodynamics.py # Thermodynamic utilities
- held_suarez.py # Held-Suarez forcing
- held_suarez_latlon.py # Held-Suarez (lat-lon)
- baroclinic_wave.py # Baroclinic wave init
- kessler.py # Legacy Kessler wrapper
- radiation/ # 2 backends + RRTMGP bundle
 - config.py, output.py, integration.py
 - gray.py # Gray two-stream
 - rrtmgp_radiation.py # Full RRTMGP wrapper
 - solar.py # Solar geometry
 - rrtmgp/ # Bundled jax-rrtmgp
 - rrtmgp.py, kernel_ops.py, interpolation.py
 - optics/ # Gas + cloud optics
 - rte/ # RTE solvers
 - config/ # RT configuration
 - utils/ # I/O utilities
- convection/ # 5 backends
 - config.py, output.py, integration.py
 - sbm.py, dca.py, kuo.py, mass_flux.py, edmf.py
- microphysics/ # 6 backends
 - config.py, output.py, integration.py
 - kessler.py, sundqvist.py, seifert_beheng.py
 - morrison.py, thompson.py, ml_emulator.py
- turbulence/ # 8 backends
 - config.py, output.py, integration.py
 - smagorinsky.py, louis.py, tke.py, clubb_lite.py
 - holtslag_boville.py, ysu.py, edmf.py, ml_emulator.py
 - surface_layer.py, vertical_diffusion.py
- gravity_wave_drag/ # 6 backends
 - config.py, output.py, integration.py
 - rayleigh.py, lindzen.py, mcfarlane.py
 - hines.py, prognostic_spectral.py, ml_emulator.py

```

ocean/
├── __init__.py
├── eos.py
├── vertical.py
├── state.py
├── init.py
├── conservation.py
├── simple_ocean.py
├── dynamics/
│   ├── ocean_model.py
│   ├── ocean_pe.py
│   ├── barotropic.py
│   ├── spectral_ocean_pe.py
│   ├── sfno_ocean.py
│   └── ocean_pe_fc_cgrid.py
├── physics/
│   ├── combined.py
│   ├── mixing.py
│   ├── vertical_mixing/
│   ├── lateral_mixing/
│   ├── surface_forcing/
│   ├── bottom_drag/
│   └── convection/
├── biogeochemistry/
│   ├── config.py
│   ├── carbon_cycle.py
│   ├── carbonate.py
│   ├── gas_exchange.py
│   └── npzd.py
└── land/
    ├── state.py
    ├── config.py
    ├── slab_land.py
    ├── multilayer_land.py
    ├── soil_grid.py
    └── soil_hydraulics.py
PDI, Lu)
├── richards.py
├── soil_thermal.py
├── carbon/
│   ├── config.py
│   ├── carbon_cycle.py
│   └── stomata.py
├── ice/
│   ├── state.py
│   └── config.py
params)
├── sea_ice.py
├── coupler/
│   ├── coupler.py
│   ├── coupling_fields.py
│   ├── config.py
│   ├── bulk_flux.py
│   ├── tile_fractions.py
│   ├── surface_exchange.py
│   ├── accumulator.py
│   └── lake/
│       ├── state.py, config.py, two_layer_lake.py
├── forcing/
└── amip.py
# Ocean component
# Wright (1997) EOS
# z-star coordinate
# OceanState, OceanConfig
# Bathymetry, initial conditions
# Volume/heat/salt fixers
# Slab ocean model
# OceanModel (split-explicit)
# Baroclinic tendencies
# Barotropic substeps
# SpectralOceanModel
# SFNOOceanModel
# FC-Gram ocean PE + div damping
# Ocean physics combiner
# Mixing utilities
# constant, richardson, kpp
# harmonic, biharmonic, gm_redi
# prescribed, restoring, bulk_formulas
# linear, quadratic
# enhanced_diffusion, plume
# Ocean carbon cycle
# BiogeoConfig, OceanBiogeoState
# Abiotic + NPZD integration
# CO2 solubility, carbonate equilibria
# Air-sea CO2 flux (Wanninkhof 2014)
# NPZD ecosystem model
# Land component
# LandState
# LandConfig, MultiLayerLandConfig
# Slab land model + stomata coupling
# Multi-layer Richards + thermal diffusion
# SoilGrid geometry
# 6 retention curves (VG, CH, BC, Campbell,
# Mixed-form Richards equation solver
# Soil thermal diffusion (Johansen 1975)
# Carbon cycle + plant physiology
# CarbonConfig, CarbonState
# DALEC-990 6-pool + seasonal schemes
# Farquhar, Ball-Berry, Medlyn, Jarvis
# Cryosphere
# SeaIceState, DynamicSeaIceState
# SeaIceConfig (dynamics, n_categories, EVP
# Slab + free-drift + EVP + multi-category
# Component coupling
# Main coupler engine
# AtmToSurface, SurfaceToAtm, TileResponse
# CouplerConfig, TileConfig
# MOST bulk flux (COARE3, LY04, fixed-z0)
# TileFractions, blend_tiles
# Atmosphere-surface fluxes
# Flux accumulation
# Two-layer lake model
# External forcing
# AMIP SST/sea-ice forcing

```

	- amip_config.py - external.py - experiments.py	# AMIPExperimentConfig, checkpoint I/O # GHG, ozone, aerosol, solar configs # CMIP6 experiment templates + GHG time
series		
	- io/ - cmor_output.py - restart.py - tuning.py - surface_albedo.py - ml/ - sfno.py - sfno_block.py - spectral_conv.py - normalization.py - channel_packing.py - conservation.py - loss.py - training.py - data/ - era5_loader.py - parallel/ - mesh.py - comm.py - distributed.py - halo_exchange.py - voronoi_partition.py	# I/O and data pipeline # CFWriter, CMOR tables, CMIP6 DRS output # RestartMetadata, reproducibility checks # Tuning parameter registry + validation # Surface albedo parameterizations # Machine learning # SFNO main class # SFNO block architecture # Spherical spectral convolution # Channel normalization # State ↔ ML channels # Conservation correction filters # Training loss functions # Training utilities # ERA5 data loader # Parallelism infrastructure # DeviceConfig, shard/replicate # CommTopology # MPI initialization # MPI halo exchange (structured grids) # Voronoi/MPAS mesh decomposition (RCB,
METIS)		
	- halo_exchange_voronoi.py - ensemble.py	# Voronoi halo exchange (MPI + simulated) # Ensemble parallelism (vmap, sharding,
scan)		
	- metal.py - reductions.py - visualization/ - maps.py - diagnostics/ __init__.py - energy_budget.py	# Apple Metal support # MPI-aware reductions # Global maps + conservation plots # Online diagnostics # Diagnostics exports # Column energy, TOA/surface flux, budget
tracker		
	- monthly_means.py - da/ - io/	# MonthlyAccumulator for zonal/global means # Data assimilation (skeleton) # I/O utilities (skeleton)
	- evaluations/ - metrics.py - weatherbench.py - visualize.py - configs/	# WeatherBench2 evaluation # RMSE, ACC, bias # WeatherBench2 framework # Evaluation visualization # Evaluation configs
	- tests/ - conftest.py - unit/ - atmosphere/	# 103 test files, 1700+ individual tests # Shared fixtures # 60+ unit test files # Atmosphere tests (dynamics, physics,
validation)		
	- ocean/ - land/	# Ocean tests (dynamics, biogeochem, MPI) # Land tests (slab, multilayer, carbon,
stability)		
	- sea_ice/ - distributed/	# Sea ice + surface albedo tests # MPI distributed + Voronoi halo tests

```

|   └─ validation/                                # Validation suite (differentiability,
stability)
|   └─ test_cases/                                # Williamson, DCMIP, DCMIP-2025
|
|   └─ scripts/                                   # Research & experiment scripts
|       └─ run_amip.py                            # AMIP simulation (gray/RRTMG, checkpoint)
|       └─ run_amip_spectral.py                   # AMIP spectral PE simulation
|       └─ run_100day.py                          # 100-day atmospheric simulation
|       └─ run_100day_spectral.py                 # 100-day spectral simulation
|       └─ run_held_suarez.py                     # Held-Suarez experiment
|       └─ run_held_suarez_latlon.py              # Held-Suarez on lat-lon
|       └─ run_held_suarez_spectral.py            # Held-Suarez spectral
|       └─ run_dycore_tests.py                   # Comprehensive dycore suite
|       └─ run_williamson_spectral.py              # Spectral Williamson tests
|       └─ run_dcmip2025.py                       # DCMIP-2025 intercomparison
|       └─ run_ocean_tests.py                    # Ocean validation
|       └─ run_ocean_realistic.py                 # Realistic ocean simulations
|       └─ run_rce_25deg.py                       # Radiative-convective equilibrium
|       └─ run_rce_slab_ocean.py                  # RCE with slab ocean
|       └─ run_rce_slab_land.py                   # RCE with slab land
|       └─ run_parallel_validation.py             # Multi-device scaling validation
|       └─ run_baroclinic_wave.py                 # Baroclinic wave experiment
|       └─ ...                                    # 26 total run scripts
|
|   └─ config/                                    # Configuration templates
|       └─ williamson_test2.yaml
|       └─ williamson_test5.yaml
|
|   └─ docs/                                       # Documentation
|       └─ REAL_HARDWARE_SCALING.md               # Multi-GPU/MPI scaling guide
|       └─ amip.md                                # AMIP experiment guide (CLI, radiation,
diagnostics)
|       └─ cmip_readiness.md                     # CMIP readiness checklist (all components)
|       └─ implementation_summary.md              # Comprehensive summary of all
implementations
|       └─ legoesm_documentation.tex              # LaTeX technical documentation (equations)
|
|   └─ notebooks/                                # Jupyter notebooks

```

Appendix A: Physical Constants

```

# legoesm/constants.py
import jax.numpy as jnp

# Fundamental
g = 9.80616                # Gravitational acceleration [m/s^2]
Omega = 7.292e-5           # Earth rotation rate [rad/s]
R_earth = 6.371229e6       # Earth mean radius [m]

# Dry air
R_d = 287.05               # Gas constant for dry air [J/(kg*K)]
c_pd = 1004.64             # Specific heat at const pressure [J/(kg*K)]
c_vd = 717.56             # Specific heat at const volume [J/(kg*K)]
kappa = R_d / c_pd         # Poisson constant (~0.2857)
p_ref = 1.0e5              # Reference pressure [Pa]

# Water
R_v = 461.51              # Gas constant for water vapor [J/(kg*K)]
c_pv = 1846.0             # Specific heat of water vapor [J/(kg*K)]
L_v = 2.501e6             # Latent heat of vaporization [J/kg]
L_s = 2.834e6             # Latent heat of sublimation [J/kg]

```

```

L_f = 3.337e5          # Latent heat of fusion [J/kg]
rho_water = 1000.0     # Density of liquid water [kg/m^3]
rho_ice = 917.0        # Density of ice [kg/m^3]

# Radiation
sigma_sb = 5.670374419e-8 # Stefan-Boltzmann constant [W/(m^2*K^4)]
S_0 = 1361.0           # Solar constant [W/m^2]

# Turbulence
kappa_vk = 0.4         # von Kármán constant

# Derived
epsilon = R_d / R_v     # Molecular weight ratio (~0.622)

```

Appendix B: Williamson Test Case 2 — Specification

Global steady-state nonlinear zonal geostrophic flow.

Initial conditions: - $u = u_0 * \cos(\text{lat})$ where $u_0 = 2\pi R_{\text{earth}} / (12 \text{ days})$ - $v = 0$ -
 $h = h_0 - (R_{\text{earth}} * \Omega * u_0 + u_0^2 / 2) * \sin(\text{lat})^2 / g$ where $h_0 = 2.94e4 / g$

Validation: After 5 days, the solution should be identical to the initial condition. Height field L2 error norm should be $< 1e-5$.

Appendix C: Williamson Test Case 5 — Specification

Zonal flow over an isolated mountain.

Initial conditions: Same as Test 2 but with: - Mountain: $h_s = h_{s0} * (1 - r/R)$ for $r < R$, where -
 $h_{s0} = 2000 \text{ m}$ - $R = \pi/9$ (20 degrees) - Center: $(3\pi/2, \pi/6)$ (30N, 90W in rotated coords) - $r = \min(R, \sqrt{(\text{lon} - \text{lon}_c)^2 + (\text{lat} - \text{lat}_c)^2})$

Run for 15 days. Validate: - Mass conservation to machine precision - Total energy conservation to $< 1e-6$
 - Qualitative comparison to published solutions

Appendix D: Summary Statistics

Category	Count
Python modules	230+
Dynamical cores	25 (3 grids × 3 equations × 5 discretizations + spectral + learned)
Physics schemes	25+ across 5 categories
Ocean physics	12 schemes across 5 categories
Bulk flux schemes	3 (MOST fixed-z0, COARE 3.0, Large & Yeager 2004)
Test files	123 (unit + integration + distributed + validation)
Test count	2500+ (all passing; 9 skipped for optional deps)

Category	Count
Research scripts	26
Grids	4 (cubed-sphere, lat-lon, Gaussian, Voronoi/MPAS)
Time integrators	6 (SSP-RK3, SSP-RK34, SSP-RK54, split-explicit, semi-implicit, leapfrog-RAW)
Discretization families	6 (C-D grid FV3, FV/PPM, spectral, FC-Gram, C-grid, MPAS/Voronoi)

End of Specification

Appendix: README

legoESM

legoESM

A Differentiable Earth System Model in JAX

legoESM is a next-generation, fully differentiable Earth System Model spanning weather-to-climate timescales. Built from scratch in JAX, it enables end-to-end gradient computation for data assimilation, parameter estimation, and hybrid AI-physics modeling.

Features

- **Differentiable-first:** End-to-end `jax.grad` through the full coupled model
- **Conservation as hard constraint:** Mass, energy, and momentum conserved via projection
- **Modular & swappable:** Standard tensor-in/tendency-out interface for AI or physics modules
- **Hardware-portable:** CPU, GPU (multi-GPU), TPU, and Apple Silicon
- **Multi-grid:** Cubed-sphere, lat-lon, Gaussian/spectral, Voronoi/MPAS, icosahedral

Model Components

Atmosphere

- **24 dynamical cores:** Shallow water, hydrostatic PE, non-hydrostatic CE across 8 discretizations (centered, FV/PPM, FC-Gram, C-grid) on cubed-sphere, lat-lon, and Gaussian grids, plus 2 SFNO learned cores
- **25+ physics schemes:** Radiation (gray + RRTMGP with diurnal cycle, ozone, cloud-radiation coupling), 5 convection backends, 6 microphysics backends, 8 turbulence backends (with PBL height diagnosis), 6 gravity wave drag backends, cloud fraction (Sundqvist, Xu-Randall)
- **Vertical coordinates:** Sigma and hybrid sigma-pressure (L20-L60, sinh stretching)

Ocean

- **3D ocean dynamics:** Split-explicit baroclinic/barotropic, spectral ocean, SFNO ocean, FC-Gram ocean
- **Ocean physics:** KPP/Richardson/constant vertical mixing, harmonic/biharmonic/GM-Redi lateral mixing, surface forcing, bottom drag, convective adjustment
- **Biogeochemistry:** Abiotic carbon cycle (DIC+ALK, carbonate equilibria, air-sea CO₂ flux) + NPZD ecosystem model
- **Simple ocean:** Slab mixed-layer and two-layer models

Land Surface

- **Slab land:** Energy balance + bucket hydrology with stomatal conductance (Farquhar + Ball-Berry/Medlyn/Jarvis)
- **Multi-layer land:** Richards equation (mixed-form Picard, 6 retention curves: VG, CH, BC, Campbell, PDI, Lu) + Johansen thermal diffusion
- **Carbon cycle:** DALEC-990 6-pool (labile/foilage/root/wood/litter/SOM) with LUE GPP + seasonal scheme
- **Snow:** Accumulation/melt budget, age-dependent albedo, latitude-varying vegetation albedo

Cryosphere

- **Sea ice:** Thermodynamic slab + free-drift + EVP rheology (Hunke & Dukowicz 1997), multi-category (Lipscomb 2001), temperature-dependent albedo

Coupler

- **Tile-based coupling:** Ocean/ice/land/lake with area-weighted blending
- **Bulk flux:** COARE 3.0, Large & Yeager 2004, fixed-z₀
- **Surface albedo:** Zenith-dependent ocean (Briegleb 1992), snow age decay, ice temperature feedback

Diagnostics

- **Energy budget:** Column moist/dry static energy, TOA/surface flux tracking, dE/dt residual monitoring
- **Monthly means:** Zonal-mean profiles, global-mean scalars, multi-year accumulation

External Forcing

- **GHG:** Constant or time-varying (NetCDF), CMIP6 experiment templates (piControl, historical, SSP2-4.5, SSP5-8.5, AMIP, 1pctCO₂)
- **Ozone/Aerosol/Solar:** Climatological or time-varying from files
- **Real topography:** NetCDF loading with bilinear regridding, Laplacian smoothing, land fraction derivation

Parallelism

- **Canonical parallel runtime** (`ParallelRuntime`): single entry point for serial, multi-GPU, MPI, and hybrid execution
- **Cubed-sphere sharding**: Face-level (1/2/3/6 devices) and sub-face tiling (6k² devices: 24, 54, 96, ...)
- **Lat-lon / level sharding**: Domain decomposition by latitude or vertical levels
- **Voronoi mesh decomposition**: Recursive coordinate bisection + METIS partitioning with halo exchange
- **Ensemble parallelism**: `vmap`-based vectorization + `NamedSharding` for multi-device, scan-based time integration with gradient checkpointing
- **MPI**: `mpi4jax`-based halo exchange and reductions for multi-node execution
- **Apple Silicon**: Metal/CPU hybrid routing

CMIP Infrastructure

- **CF-compliant output**: `CFWriter` with CF-1.8, CMIP6 DRS naming, 27 variables (Amon + Lmon)
- **Restart/reproducibility**: SHA-256 state digests, config hashes, platform metadata
- **Tuning**: 16-parameter registry with resolution-appropriate defaults

Quick Start

```
# Install
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"

# Run Williamson Test Case 2
legoesm test williamson --case 2 --resolution 48 --days 5

# Run tests
pytest tests/
```

Platform Notes

Compatibility Matrix

Install minimum (from `pyproject.toml`): Python ≥ 3.11 , JAX $\geq 0.4.35$, `mpi4jax` $\geq 0.8, < 0.9$ (optional), `mpi4py` $\geq 4.1, < 5$ (optional).

Tested range — the versions CI and benchmarks run against:

Path	Hardware / Backend	MPI Runtime	Tested JAX	Tested <code>mpi4jax</code>	Status / Notes
Finite-volume dycores + ocean (single-process)	CPU (<code>jax</code> CPU backend)	N/A	<code>>=0.8, <0.10</code>	N/A	Regular unit/regression path

Path	Hardware / Backend	MPI Runtime	Tested JAX	Tested mpi4jax	Status / Notes
Finite-volume dycores + ocean (single-process)	Apple Silicon Metal (<code>jax-metal</code>)	N/A	<code>>=0.8, <0.10</code>	N/A	FV solvers only (<code>float32</code>); no <code>float64</code>
Spectral solvers (atmosphere/ ocean)	CPU (<code>JAX_PLATFORMS=cpu</code>)	N/A	<code>>=0.8, <0.10</code>	N/A	Requires <code>float64</code> / <code>complex128</code> ; not Metal-compatible
Distributed MPI halo/ reductions	CPU + OpenMPI (<code>mpirun</code>)	OpenMPI 4.x/5.x	<code>>=0.8, <0.10</code>	<code>>=0.8, <0.9</code>	Validated with <code>mpirun -np 2/3/6</code>
Multi-device scaling suite	CPU/GPU (if available)	Optional	<code>>=0.8, <0.10</code>	<code>>=0.8, <0.9</code> (MPI mode)	<code>scripts/run_parallel_validation.py</code>

JAX versions outside the tested range may work but are not guaranteed. Versions below the install minimum will fail at `pip install`.

`legoesm.parallel.reductions` enforces MPI compatibility guardrails at runtime: - **Hard error** for `mpi4jax<0.8` (incompatible token semantics). - **Warning** for JAX or `mpi4jax` outside the tested range. Set `LEGOESM_MPI_STRICT_COMPAT=1` to promote the warning to a hard error.

Detailed runbook for real hardware MPI/multi-GPU scaling: - [docs/REAL_HARDWARE_SCALING.md](#)

Parallel Runtime & Supported Device Counts

The canonical entry point for all parallelism is `ParallelRuntime.create()`:

```
from legoesm.parallel import ParallelRuntime
rt = ParallelRuntime.create(grid_type="cubed_sphere", grid_n=48)
```

Supported cubed-sphere device/rank counts (others are rejected with a precise error): - **Face-only**: 1, 2, 3, 6 - **Sub-face tiling**: $6k^2$ for $k \geq 2$: 24, 54, 96, 150, 216, 294, 384, 600, ...

Unsupported counts (4, 5, 7, 8, 12, 36, 48, ...) raise `ValueError` with the nearest valid counts. There is no silent round-down.

Execution modes: | Mode | Ranks | Devices/rank | Halo backend | Reduction backend |
|-----|-----|-----|-----|-----|-----| | `serial` | 1 | 1 | local | local | | `multi_device` | 1 | N |
JAX SPMD | local | | `mpi` | N | 1 | MPI | MPI | | `hybrid` | N | M | MPI + JAX | MPI + JAX |

Deprecated APIs: `partition_state()` (zero-masked global arrays) emits `DeprecationWarning`. Use `ParallelRuntime.scatter()` / `.gather()` instead.

Apple Silicon (Metal/MPS backend)

The **spectral solver** (Gaussian grid + spherical harmonic transforms) requires `float64` and `complex128` arithmetic, which Apple's Metal backend does not support.

If you are on Apple Silicon and want to use the spectral solver, force the CPU backend:

```
JAX_PLATFORMS=cpu python your_script.py
```

The **finite-volume solvers** (cubed-sphere shallow water, hydrostatic primitive equations) work on all backends including Metal, using `float32` precision.

To check your current JAX backend:

```
python -c "import jax; print(jax.default_backend())"
```

Documentation

- [SPECIFICATION.md](#) — Full technical specification (v3.8)
- [docs/implementation_summary.md](#) — Comprehensive summary of all implementations and tests
- [docs/cmip_readiness.md](#) — CMIP production readiness checklist
- [docs/amip.md](#) — AMIP experiment guide
- [docs/REAL_HARDWARE_SCALING.md](#) — Multi-GPU/MPI scaling guide

Acknowledgments

legoESM bundles [jax-rrtmgp](#) (Apache 2.0 license) for correlated-k radiation. `jax-rrtmgp` was developed by Jeff Parker (Google), Duncan Watson-Parris (UCSD), and Juan Nathaniel (Columbia University).

License

MIT

Appendix: Implementation Summary

legoESM — Implementation Summary

Last updated: 2026-03-23

Overview

legoESM is a fully differentiable Earth System Model implemented in JAX, comprising atmosphere, ocean, land surface, cryosphere, and coupler components. The model supports 24 atmospheric dynamical cores

across 8 discretizations, 25+ physics parameterizations, multi-layer land/ocean, sea ice with EVP dynamics, and full CMIP infrastructure.

1. Atmosphere Dynamics

Dynamical Cores (24 total)

Equations	Discretization	Grid	File
Shallow Water	Centered	Cubed-sphere	<code>shallow_water.py</code>
Shallow Water	FV/PPM	Cubed-sphere	<code>shallow_water_fv.py</code>
Shallow Water	FV/PPM	Lat-lon	<code>shallow_water_fv_latlon.py</code>
Shallow Water	FC-Gram	Cubed-sphere	<code>shallow_water_fc.py</code>
Shallow Water	FC-Gram C-grid	Cubed-sphere	<code>shallow_water_fc_cgrid.py</code>
Shallow Water	C-grid	Lat-lon	<code>shallow_water_cgrid_latlon.py</code>
Shallow Water	Spectral	Gaussian	<code>spectral_sw.py</code>
Shallow Water	SFNO (learned)	Cubed-sphere	<code>sfno_sw.py</code>
Hydrostatic PE	Centered	Cubed-sphere	<code>primitive_eq.py</code>
Hydrostatic PE	FV/PPM	Cubed-sphere	<code>primitive_eq_fv.py</code>
Hydrostatic PE	Centered	Lat-lon	<code>primitive_eq_latlon.py</code>
Hydrostatic PE	FV/PPM	Lat-lon	<code>primitive_eq_fv_latlon.py</code>
Hydrostatic PE	FC-Gram	Cubed-sphere	<code>primitive_eq_fc.py</code>
Hydrostatic PE	FC-Gram C-grid	Cubed-sphere	<code>primitive_eq_fc_cgrid.py</code>
Hydrostatic PE	Spectral	Gaussian	<code>spectral_pe.py</code>
Hydrostatic PE	SFNO (learned)	Cubed-sphere	<code>sfno_pe.py</code>
Non-hydrostatic CE	Centered	Cubed-sphere	<code>compressible_euler.py</code>
Non-hydrostatic CE	FV/PPM	Cubed-sphere	<code>compressible_euler_fv.py</code>
Non-hydrostatic CE	FV/PPM	Lat-lon	<code>compressible_euler_fv_latlon.py</code>
Non-hydrostatic CE	FC-Gram	Cubed-sphere	<code>compressible_euler_fc.py</code>
Non-hydrostatic CE	FC-Gram C-grid	Cubed-sphere	<code>compressible_euler_fc_cgrid.py</code>
Non-hydrostatic CE	C-grid	Cubed-sphere	<code>compressible_euler_cgrid.py</code>
Non-hydrostatic CE	Spectral	Gaussian	<code>spectral_nh.py</code>
MPAS SW	Voronoi	Unstructured	(via MPAS state)

Vertical Coordinates

- **Sigma coordinate** (`SigmaCoordinate`): Standard terrain-following
- **Hybrid sigma-pressure** (`HybridSigmaPressureCoordinate`): Factory functions `make_hybrid_levels()`, `standard_hybrid_levels()`, `hybrid_from_sigma()`
- **Sinh stretching**: Boundary-layer concentration via `stretching` parameter
- **Resolution**: L20 ($p_{\text{top}}=1000\text{Pa}$), L40 ($p_{\text{top}}=200\text{Pa}$, default), L60 ($p_{\text{top}}=10\text{Pa}$)

Grid Infrastructure

- **Cubed-sphere**: 6-face gnomonic projection, C16-C64 tested
- **Lat-lon**: Regular grid with polar filter
- **Gaussian**: Gauss-Legendre quadrature + spherical harmonic transforms
- **Voronoi/MPAS**: Unstructured dual mesh (triangles + hexagons)
- **Halo exchange**: Cubed-sphere face-to-face, lat-lon periodic, Voronoi entity-based
- **Topography**: Real NetCDF loading, bilinear regridding, Laplacian smoothing, land fraction derivation

2. Atmosphere Physics (25+ schemes)

Radiation (2 backends)

- **Gray two-stream** (`gray.py`): Held-Suarez-like OLR
- **RRTMGP** (`rrtmgp_radiation.py`): Full correlated-k gas + cloud optics
- **Diurnal cycle**: Optional $\cos(\text{SZA})$ insolation with hour-angle and declination
- **Ozone**: Standard (US Std Atm 1976), analytical (Gaussian in $\log p$), or none
- **Cloud-radiation coupling**: Cloud fraction $\rightarrow$ LWP/IWP $\rightarrow$ cloud optics $\rightarrow$ RRTMGP

Cloud Fraction (2 schemes)

- **Sundqvist**: RH-only threshold scheme
- **Xu-Randall**: RH + explicit condensate, pressure-dependent exponent
- Diagnostic condensate partitioned by temperature when no microphysics active

Convection (5 backends)

- SBM (Simplified Betts-Miller), DCA, Kuo, mass flux, EDMF

Microphysics (6 backends)

- Kessler, Sundqvist, Seifert-Beheng, Morrison, Thompson, ML emulator
- Operator-split integration in AMIP: called after convection, before BL exchange
- q_c/q_r managed as parallel arrays; saturation adjustment disabled when active

Turbulence (8 backends)

- Smagorinsky, Louis, TKE, CLUBB-lite, Holtslag-Boville, YSU, EDMF, ML emulator

- **PBL height diagnosis** (`pbl_height.py`): Bulk Richardson number, sigmoid-weighted or linear interpolation to `Ri_crit` crossing
- All backends return `h_pbl` in `TurbulenceOutput`

Gravity Wave Drag (6 backends)

- Rayleigh, Lindzen, McFarlane, Hines, prognostic spectral, ML emulator

Combined Physics

- `combined.py` : Sums tendencies from all active physics modules
- Factory pattern: `make_*_physics()` returns `physics_fn(state, grid, coords)`
- Time-dependent: `set_time(day_of_year, seconds_of_day)` method on closures

3. Ocean

Dynamics

- **3D split-explicit** (`ocean_model.py`): Baroclinic PE + barotropic substeps
- **Spectral ocean** (`spectral_ocean_pe.py`): Spherical harmonic ocean
- **SFNO ocean** (`sfno_ocean.py`): Learned ocean dynamics
- **FC-Gram ocean** (`ocean_pe_fc_cgrid.py`): High-order ocean PE
- **Simple ocean**: Slab mixed-layer (`simple_ocean.py`) and two-layer

Physics

- Vertical mixing: constant, Richardson, KPP
- Lateral mixing: harmonic, biharmonic, GM-Redi
- Surface forcing: prescribed, restoring, bulk formulas
- Bottom drag: linear, quadratic
- Convection: enhanced diffusion, plume

Biogeochemistry

- **Abiotic** (`carbon_cycle.py`): DIC + ALK, carbonate equilibria, air-sea CO₂ flux (Wanninkhof 2014)
- **NPZD** (`npzd.py`): Nutrient-Phytoplankton-Zooplankton-Detritus ecosystem + Redfield coupling
- EOS: Wright (1997), z-star vertical coordinate

4. Land Surface

Slab Land (`slab_land.py`)

- Energy balance: $R_{net} = SH + LH + G$
- Bucket hydrology: precipitation, evaporation, runoff
- Stomatal conductance: Farquhar photosynthesis + Ball-Berry/Medlyn/Jarvis

Multi-Layer Land (`multilayer_land.py`)

- **Richards equation** (`richards.py`): Mixed-form Picard iteration (Celia 1990), Thomas algorithm, surface/subsurface runoff
- **6 retention curves** (`soil_hydraulics.py`): van Genuchten, Clapp-Hornberger, Brooks-Corey, Campbell, Peters-Durner-Iden (2015), Lu (2016)
- **Soil thermal** (`soil_thermal.py`): Johansen (1975) conductivity, backward Euler diffusion
- **Soil grid** (`soil_grid.py`): Geometric growth factor, configurable `n_layers/dz_top/total_depth`

Carbon Cycle (`carbon/`)

- **DALEC-990**: 6-pool model (labile/foilage/root/wood/litter/SOM), LUE-based GPP, Q10 decomposition, DALEC phenology
- **Seasonal**: Sinusoidal NEE with latitude-dependent amplitude/phase
- Integrated into both slab and multi-layer land (returns 3-tuple: state, response, carbon_state)

Snow & Albedo (`surface_albedo.py`)

- Snow accumulation from precipitation, melt above `T_melt`
- Age-dependent snow albedo decay
- Latitude-varying vegetation albedo (tropics/midlat/highlat)
- All feedback optional and disabled by default

5. Cryosphere

Sea Ice (`sea_ice.py`)

- **Thermodynamic slab**: Growth/melt, conductive flux
- **Free drift**: Diagnostic ice velocity
- **EVP rheology**: Hunke & Dukowicz (1997), subcycled momentum solver
- **Multi-category**: CICE framework, Lipscomb (2001) linear remapping
- **Temperature-dependent albedo**: Cold/warm transition via `IceAlbedoConfig`

6. Coupler

Surface Coupling (`coupler.py`)

- Tile-based: ocean, ice, land, lake with area-weighted blending
- Flux accumulator with window-mean and residual carry
- `q_surface` recomputed from updated T in all tiles

Bulk Flux (`bulk_flux.py`)

- COARE 3.0, Large & Yeager 2004, fixed-z0
- Monin-Obukhov surface layer theory

Lake (lake/)

- Two-layer model: epilimnion + hypolimnion with wind-enhanced mixing
-

7. Diagnostics

Energy Budget (diagnostics/energy_budget.py)

- `column_moist_static_energy()` : Integrates $c_p T + L_v q + \Phi + 0.5 v^2$ via `jax.lax.scan`
- `toa_net_radiation()`, `surface_net_radiation()`, `surface_energy_flux()`
- `EnergyBudgetTracker` : Timeseries accumulation, dE/dt computation, residual $R_{TOA} - dE/dt$

Monthly Means (diagnostics/monthly_means.py)

- `MonthlyAccumulator` : Bins 2D fields by latitude (zonal means), accumulates 3D profiles, tracks global-mean scalars
 - 365-day calendar month mapping, save/load to npz
-

8. External Forcing

- **GHG**: `GHGConfig` with CO₂/CH₄/N₂O, constant or time-varying from NetCDF
 - **Ozone**: Monthly zonal-mean climatology + analytical profiles
 - **Aerosol**: AOD climatology from NetCDF
 - **Solar**: TSI constant or time-varying
 - **CMIP6 experiments**: piControl, historical, SSP2-4.5, SSP5-8.5, AMIP, 1pctCO₂ templates with built-in GHG time series
 - **Topography**: Real NetCDF loading, bilinear regrid, Laplacian smoothing, edge blending
-

9. Parallelism

Structured Grid Sharding (parallel/mesh.py)

- Cubed-sphere face sharding (1-6 devices)
- Sub-face tiling (>6 devices, multiples of 6)
- Lat-lon domain decomposition by latitude
- Level-parallel spectral transforms

Voronoi Mesh Decomposition (parallel/voronoi_partition.py)

- Recursive Coordinate Bisection (RCB) partitioner
- METIS graph partitioner (optional)
- 2-layer halo depth, entity ownership rules
- `VoronoiPartition`, `HaloCommSchedule` data structures
- `build_local_mesh()`, `scatter_to_local()` utilities

Voronoi Halo Exchange (`parallel/halo_exchange_voronoi.py`)

- `VoronoiHaloExchange` class with MPI and simulated backends
- `mpi4jax` sendrecv with entity-type tag offsets
- Global-to-local index remapping, connectivity masking

Ensemble Parallelism (`parallel/ensemble.py`)

- `stack_states()` / `unstack_states()` : Batch/unbatch pytree states
- `perturb_initial_conditions()` / `perturb_parameters()` : Stochastic ensemble generation
- `make_ensemble_step()` : vmap wrapper for single-device vectorization
- `ensemble_integrate()` : scan-outside-vmap-inside architecture (one compiled loop for all members)
- `ensemble_integrate_with_forcing()` : Time-varying external forcing support
- `create_ensemble_mesh()` / `shard_ensemble()` / `gather_ensemble()` : Multi-device `NamedSharding`
- Statistics: `ensemble_mean()` , `ensemble_std()` , `ensemble_percentile()` , `ensemble_spread()`
- Gradient checkpointing via `jax.checkpoint` for $O(\sqrt{n})$ memory in reverse-mode AD

MPI (`parallel/distributed.py` , `halo_exchange.py` , `reductions.py`)

- `mpi4jax`-based halo exchange and global reductions
- MPI version compatibility guardrails

Apple Silicon (`parallel/metal.py`)

- Metal/CPU hybrid routing for mixed-precision workloads

10. CMIP Infrastructure

- **CF-compliant output** (`io/cmor_output.py`) : CF-1.8, CMIP6 DRS naming, 27 variables (Amon + Lmon)
- **Restart** (`io/restart.py`) : SHA-256 state digests, config hashes, platform metadata
- **Tuning** (`tuning.py`) : 16-parameter registry with resolution-appropriate defaults

11. AMIP Simulations

Scripts

- `run_amip.py` : Full-featured AMIP (cubed-sphere), supports gray/RRTMGP radiation, all physics, checkpointing, energy budget tracking, monthly means
- `run_amip_spectral.py` : Spectral PE AMIP (Gaussian grid)
- `run_amip_production.sh` : Launcher for C48/L40 10-year production
- `validate_amip.py` : Post-processing validation against observational targets

CLI Flags (run_amip.py)

`--resolution`, `--nlev`, `--days`, `--dt`, `--radiation`, `--diurnal-cycle`, `--ozone-source`, `--clouds`, `--microphysics`, `--topography`, `--vertical-coord`, `--p-top`, `--stretching`, `--dynamic-albedo`, `--monthly-means`, `--carbon-cycle`

Validated Configurations

- 365-day AMIP at C16/L40 (cubed-sphere) and T21/L40 (spectral)
- Production target: 10-year AMIP at C48/L40

12. Test Suite

123 test files, 2500+ individual tests across 10 test directories. All tests pass (0 failures; 9 skipped for optional dependencies).

Test Categories

Category	Files	Tests	Description
Core unit tests	~15	~200	Field, grid, operators, halo, conservation, timestepping
Atmosphere dynamics	~25	~400	All 24 dynamical cores, FV integration, spectral
Atmosphere physics	~10	~250	Radiation (diurnal, ozone, clouds), convection, microphysics, turbulence (PBL), GWD
Ocean	~8	~150	Dynamics, biogeochem, MPI conservation, FC-Gram, FV
Land surface	~5	~180	Slab, multilayer, carbon cycle, stomata, stability
Sea ice / albedo	~3	~80	EVP dynamics, multi-category, surface albedo
Coupler / forcing	~5	~100	Bulk flux, tile coupling, AMIP config, external forcing
Parallelism	~5	~80	Device mesh, Voronoi partition/halo (34 tests), ensemble (32 tests)
Diagnostics	~3	~50	Energy budget, monthly means
Validation	~8	~100	Differentiability, eigenvalues, stability, held-suarez
CMIP infrastructure	~3	~50	CMOR output, experiments, restart
DCMIP test cases	~6	~30	Williamson, DCMIP-2025 cases 1-3

Key Test Files (recent additions)

File	Tests	Runtime	Description
<code>test_radiation.py</code>	26+	~30s	Diurnal cycle, ozone profiles, cloud fraction, cloud-radiation
<code>test_hybrid_vertical.py</code>	34	~10s	Hybrid coord construction, pressure, geopotential, semi-implicit
<code>test_topography.py</code>	27	~15s	Regriidding, land fraction, smoothing, NetCDF, edge blending

File	Tests	Runtime	Description
test_turbulence.py	10+	~20s	PBL height (Ri, shape, bounds, interp, all backends)
test_surface_albedo.py	25	~10s	Vegetation, snow, ice, ocean albedo
test_energy_budget.py	25	~15s	Column energy, TOA/surface flux, tracker
test_monthly_means.py	21	~10s	Day-to-month, 2D/3D/scalar accumulation, save/load
test_multilayer_land.py	49	~40s	Grid, 6 retention curves, Richards, thermal, integrated
test_carbon_cycle.py	43	~30s	DALEC-990, GPP, seasonal, coupler integration
test_microphysics.py	8	~15s	Checkpoint with hydrometeors, config, multi-step stability
test_voronoi_halo.py	34	~36s	Partitioning (13), local mesh (4), operators (8), halo exchange (9)
test_ensemble.py	32	~10s	Batching (3), perturbation (8), step (4), integrate (5), statistics (5), sharding (4)
test_land_stability.py	32	~720s	Slab (6), multilayer (7), energy/water budget, snow, carbon, seasonal

Running Tests

```
# All tests (requires float64)
JAX_ENABLE_X64=1 pytest tests/

# Specific component
JAX_ENABLE_X64=1 pytest tests/land/
JAX_ENABLE_X64=1 pytest tests/unit/test_ensemble.py

# MPI distributed tests
mpirun -np 6 python -m pytest tests/distributed/test_halo_mpi.py
```

13. Implementation Roadmap (Completed)

All 12 tasks from `NEXT_STEPS.md` are complete:

1. **Diurnal Cycle** — cos(SZA) insolation, hour-angle, `set_time()` mechanism
2. **Prescribed Ozone** — Standard/analytical profiles, RRTMGP integration
3. **Cloud Fraction + Cloud-Radiation** — Sundqvist/Xu-Randall, diagnostic condensate, RRTMGP cloud optics
4. **Hybrid Sigma-Pressure Coordinate** — Full vertical coordinate with semi-implicit support
5. **Vertical Resolution L40-L60** — Sinh stretching, `standard_hybrid_levels()`
6. **Real Topography** — NetCDF loading, bilinear regrid, Laplacian smoothing
7. **Microphysics in AMIP** — Operator-split, `q_c/q_r` parallel arrays, checkpoint extension
8. **Multi-Layer Soil** — Richards equation (6 retention curves), Johansen thermal
9. **PBL Height Diagnosis** — Bulk Richardson, all 8 backends updated
10. **Surface Albedo** — Snow age, vegetation latitude, ice temperature, ocean zenith
11. **Energy Budget Closure** — Column MSE, TOA/surface flux, budget tracker
12. **10-Year AMIP at C48/L40** — Monthly means, validation script, production launcher

Additional completed features beyond the roadmap: - **Land Carbon Cycle** — DALEC-990 6-pool + seasonal, integrated into slab/multilayer - **Voronoi Mesh Decomposition** — RCB + METIS partitioning, halo exchange - **Ensemble Parallelism** — vmap + sharding + scan architecture - **Land Stability Validation** — 32 comprehensive tests (energy/water budget, seasonal behavior)

Appendix: CMIP Readiness

CMIP Readiness Status

Status of legoESM components for CMIP-class production experiments.

Surface Coupler

Feature	Status	Notes
Tile-based coupling (ocean/ice/land/lake)	Done	Area-weighted blending, flux accumulator
Ocean albedo (constant)	Done	<code>CouplerConfig.ocean_albedo</code> honoured
Ocean albedo (zenith-dependent)	Done	Briegleb 1992 via <code>OceanAlbedoConfig(method="zenith")</code>
Snow albedo feedback	Done	Age-dependent, latitude-varying
Ice albedo (temperature-dependent)	Done	Via <code>SeaIceConfig.temp_dependent_albedo</code>
Bulk flux schemes	Done	Constant, COARE 3.0, Large & Yeager 2004
q_surface consistency	Done	Recomputed from updated T in all slab tiles
Coupling accumulator / asynchronous coupling	Done	Window-mean with residual carry

External Forcing

Feature	Status	Notes
GHG concentrations (constant)	Done	CO2, CH4, N2O via <code>GHGConfig</code>
GHG concentrations (time-varying from file)	Done	NetCDF interpolation in <code>get_ghg_at_time()</code>
Ozone climatology (monthly zonal-mean)	Done	NetCDF loading + cyclic interpolation
Aerosol optical depth climatology	Done	NetCDF loading + cyclic interpolation
Total solar irradiance (constant)	Done	Via <code>SolarConfig.S_0</code>
Total solar irradiance (time-varying from file)	Done	NetCDF interpolation in <code>get_tsi_at_time()</code>
Ozone connected to radiation	Done	

Feature	Status	Notes
		Analytical + standard profiles, passed to RRTMGP via <code>o3_vmr</code>
Aerosol connected to radiation	Done	AOD climatology loaded and applied
Volcanic forcing	Done	Stratospheric AOD perturbation support
Land-use change forcing	Missing	Static land fraction only

Radiation

Feature	Status	Notes
Gray radiation (Held-Suarez-like)	Done	
RRTMGP (multi-band)	Done	Gas optics, cloud optics
Diurnal cycle	Done	Optional via <code>RadiationConfig.diurnal_cycle</code>
Spectral solar distribution	Missing	Single TSI value only, no spectral bands
Interactive aerosol-radiation coupling	Missing	

Atmosphere Dynamics

Feature	Status	Notes
Cubed-sphere (hydrostatic, nonhydrostatic)	Done	C16-C64 tested
Spectral transform (T21-T85)	Done	
Centered, FV, FC-Gram, FC-Gram C-grid discretizations	Done	
Hyperdiffusion	Done	
Sponge layer	Done	

Ocean

Feature	Status	Notes
Fixed SST (AMIP-style)	Done	Constant or spatial map
Slab ocean (mixed layer)	Done	Freezing clamp
Two-layer ocean	Done	Deep restoring option
3D ocean dynamics (multiple discretizations)	Done	Centered, FV, FC-Gram, FC-Gram C-grid
Ocean biogeochemistry (abiotic)	Done	DIC + ALK, carbonate equilibria, air-sea CO ₂ (Wanninkhof 2014)
Ocean biogeochemistry (NPZD)	Done	N-P-Z-D ecosystem + Redfield coupling to DIC/ALK
Ocean tracer transport	Done	FV tracer option

Sea Ice

Feature	Status	Notes
Thermodynamic slab ice	Done	Growth/melt, conductive flux
Ice concentration (prognostic)	Done	
Ice velocity (free drift)	Done	Diagnostic
EVP rheology / dynamics	Done	Hunke & Dukowicz 1997, subcycled momentum, <code>dynamics="evp"</code>
Multi-category ice	Done	CICE framework, Lipscomb 2001 linear remapping, <code>n_categories > 1</code>

Land Surface

Feature	Status	Notes
Slab land (thermal + bucket hydrology)	Done	
Multilayer land	Done	Richards equation (6 retention curves) + Johansen thermal diffusion
Snow budget (accumulation/melt)	Done	Age-dependent albedo, latitude-varying vegetation
Vegetation / stomatal conductance	Done	Farquhar + Ball-Berry/Medlyn (coupled), Jarvis (uncoupled)
Land carbon cycle	Done	DALEC-990 6-pool + seasonal scheme; GPP via Farquhar or LUE
Land stability validation	Done	32 tests: energy/water budget closure, seasonal behavior, 180-day runs

Lake

Feature	Status	Notes
Two-layer lake (epilimnion + hypolimnion)	Done	Wind-enhanced mixing

Diagnostics

Feature	Status	Notes
Energy budget tracking	Done	Column MSE, TOA/surface fluxes, dE/dt residual
Monthly-mean accumulation	Done	Zonal means, 3D profiles, global scalars
AMIP validation targets	Done	T_2m, precip, OLR, TOA imbalance, jet position, ITCZ

Parallelism

Feature	Status	Notes
Cubed-sphere face sharding	Done	1-6 devices

Feature	Status	Notes
Sub-face tiling	Done	Multiples of 6 devices
Lat-lon / level sharding	Done	Domain decomposition
MPI halo exchange (structured)	Done	mpi4jax, validated with <code>mpirun -np 2/3/6</code>
Voronoi mesh decomposition	Done	RCB + METIS, 2-layer halo, entity ownership
Voronoi halo exchange (MPI)	Done	mpi4jax sendrecv, entity-type tagging
Ensemble parallelism	Done	vmap + NamedSharding + scan, gradient checkpointing
Apple Silicon Metal	Done	Metal/CPU hybrid routing

CMIP Infrastructure

Feature	Status	Notes
CMOR/CF-compliant output	Done	<code>CFWriter</code> with CF-1.8, CMIP6 DRS naming, 27 variables (Amon + Lmon)
Experiment templates	Done	piControl, historical, SSP2-4.5, SSP5-8.5, AMIP, 1pctCO ₂
Built-in GHG time series	Done	Linear interpolation for historical + SSP scenarios
Restart/reproducibility	Done	SHA-256 state digests, config hashes, platform metadata
Tuning guide	Done	16 parameters, validation, resolution-appropriate defaults

Remaining Gaps for CMIP Production

1. **Land-use change:** Static land mask, no transient land cover.
2. **Dynamic vegetation:** No interactive LAI (currently prescribed).
3. **Spectral solar distribution:** Only broadband TSI, no spectral bands.
4. **Interactive aerosol-cloud coupling:** AOD affects radiation but not cloud droplet number concentration.
5. **Ice sheet dynamics:** No shallow-ice/shallow-shelf approximation.
6. **Atmospheric chemistry:** No prognostic CH₄/N₂O/CFC/O₃ tracers (prescribed only).